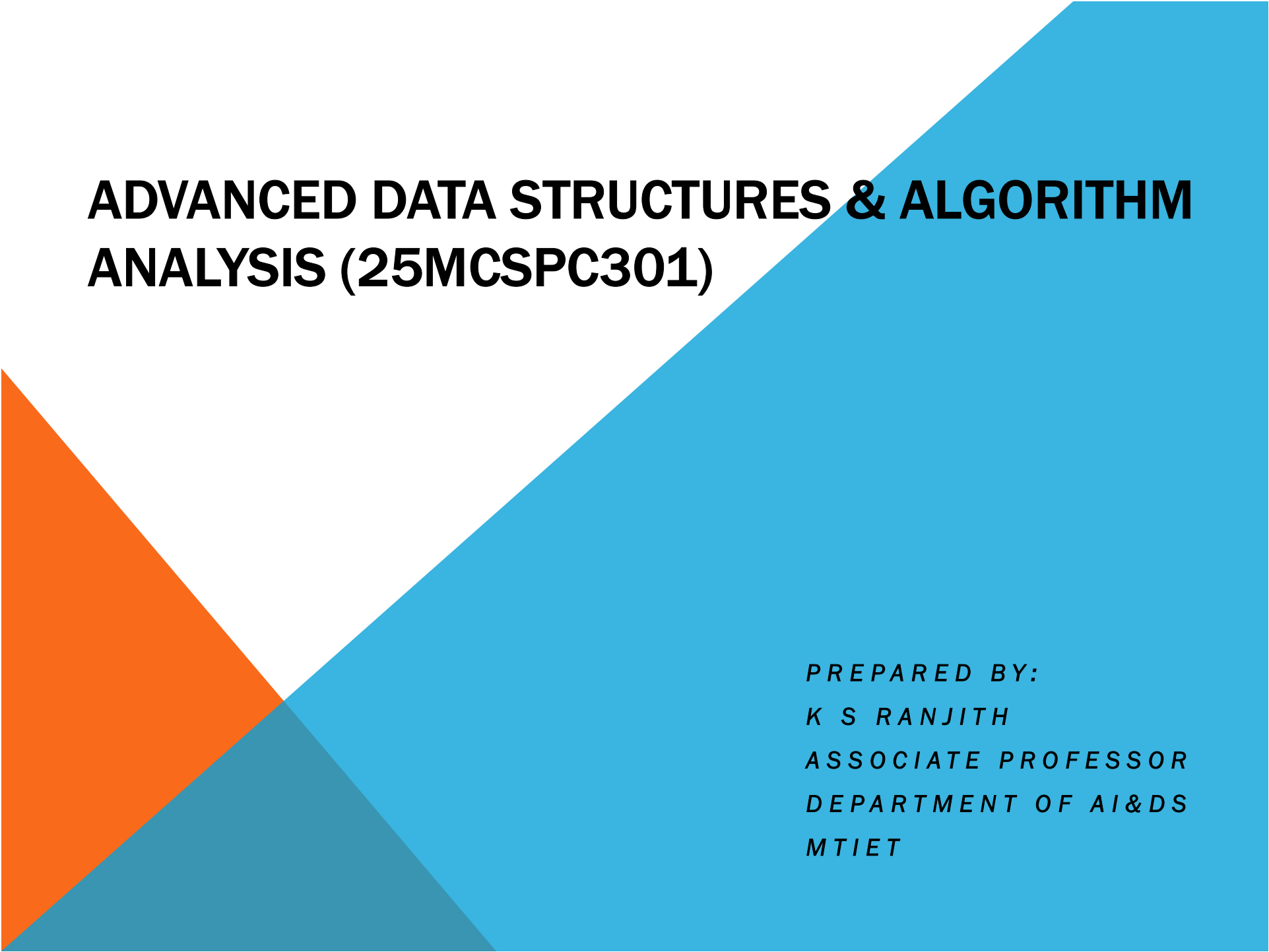




ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS (25MCSPC301)

*PREPARED BY:
K S RANJITH
ASSOCIATE PROFESSOR
DEPARTMENT OF AI&DS
MTIET*

UNIT 2

Heap Trees (Priority Queues) – Min and Max Heaps, Operations and Applications

Graphs – Terminology, Representations, Basic Search and Traversals, Connected Components and Biconnected Components, applications

Divide and Conquer: The General Method, Quick Sort, Merge Sort, Strassen's matrix multiplication, Convex Hull

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

HEAP TREES (PRIORITY QUEUES)

Introduction

A heap data structure is a specialized **binary tree** used in DSA to efficiently manage **priority queues**. Here, we will discuss the basics of a heap tree in data structure, its **types**, and its significance in various **algorithms**,

COMPLETE BINARY TREE

Complete Binary Tree:

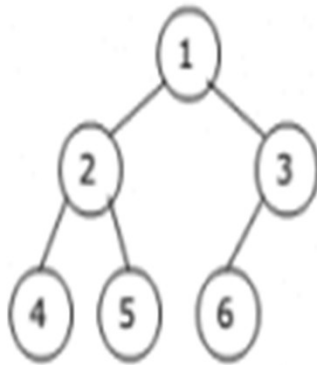
A **complete binary tree** is a type of binary tree where **all levels are completely filled except possibly the last**, and in the last level, nodes are filled **from left to right without gaps**.

Key Properties:

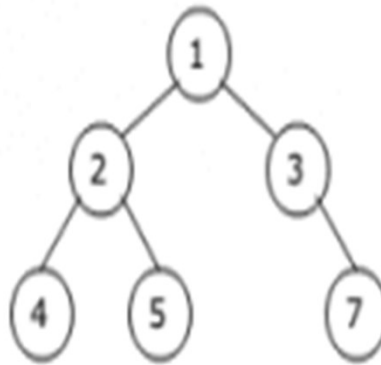
- All levels except the last are full.
- Nodes in the last level are aligned to the left.
- Height of a complete binary tree with n nodes is $\lfloor \log_2(n) \rfloor$.
- Number of nodes at depth d is 2^d .

COMPLETE BINARY TREE

Example of Complete Binary tree



Complete Binary Tree
but not strict
(a)



Not Complete and not
strict
(b)



Not Complete and not
strict
(c)

DEFINITION

What is Heap Data Structure?

A Heap is a **complete binary tree** data structure that satisfies the heap property where each parent node **is greater than or equal to** its child nodes in a **max-heap** or **less than or equal to** its child nodes in a **min heap**. This means the **largest or smallest value** is always at the **top (root)** of the tree.

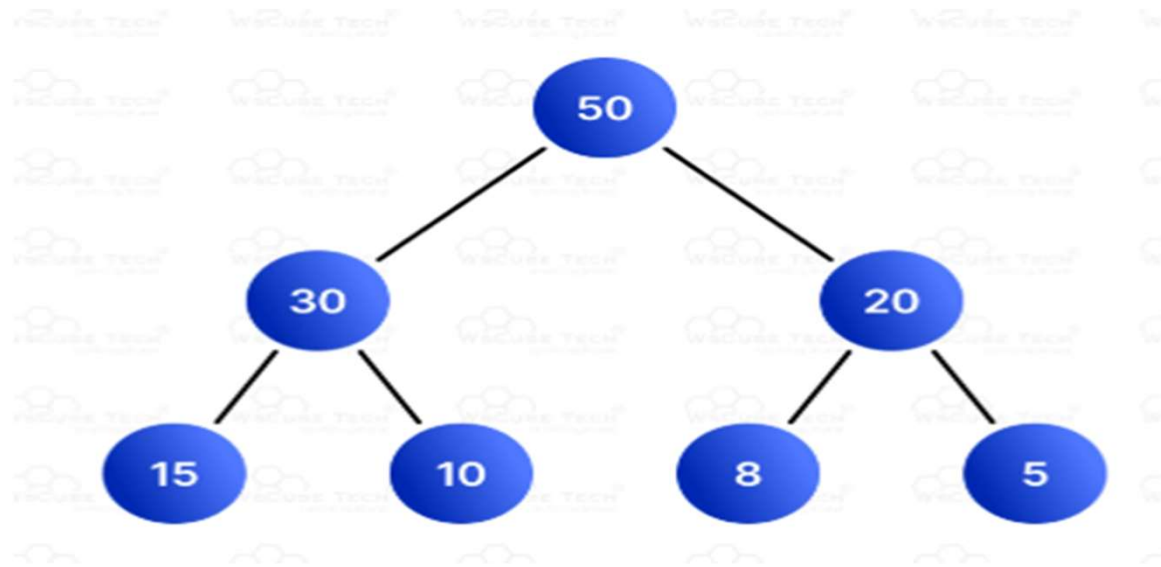
TYPES OF HEAP DATA STRUCTURE (WITH EXAMPLE)

There are two main types of heaps: Max-Heap and Min-Heap in data Structure

1. Max-Heap

In a max-heap, each parent node is **greater than or equal to** its child nodes. This ensures that the **largest value** is always at the **root** of the tree.

EXAMPLE OF A MAX-HEAP:



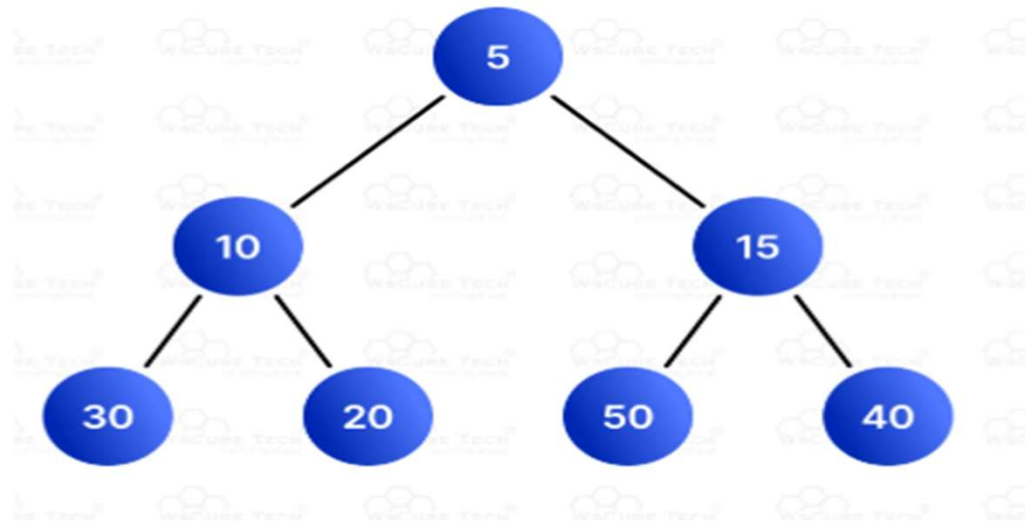
- The root node (50) is greater than its children (30 and 20).
- Each parent node (30, 20) is greater than its children (15, 10 for 30 and 8, 5 for 20).

TYPES OF HEAP DATA STRUCTURE (WITH EXAMPLE)

2. Min-Heap

In a min-heap, each parent node is **less than or equal** to its child nodes. This ensures that the **smallest value** is always at the **root** of the tree.

EXAMPLE OF A MIN-HEAP:



- The root node (5) is smaller than its children (10 and 15).
- Each parent node (10, 15) is smaller than its children (30, 20 for 10 and 50, 40 for 15).

PROPERTIES OF HEAP DATA STRUCTURE

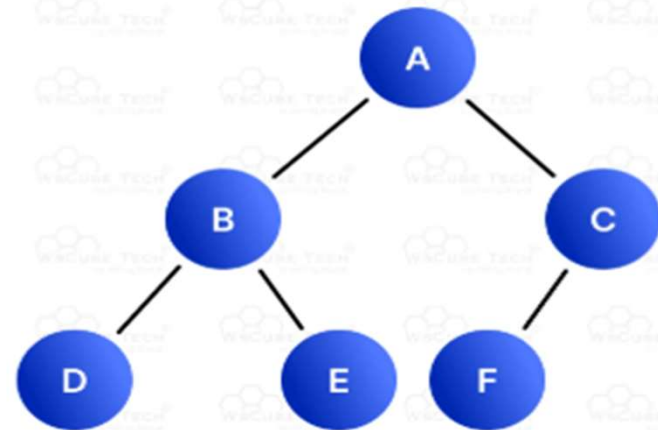
1. Complete Binary Tree

A heap is always a complete binary tree, meaning:

- All levels are fully filled except possibly the last level.
- The last level is filled from left to right.

This property ensures that the tree remains balanced, which is crucial for maintaining the efficiency of heap operations.

In this example, all levels are fully filled except the last level, which is filled from left to right.



PROPERTIES OF HEAP DATA STRUCTURE

2. Heap Order Property

The heap must satisfy the heap order property, which differs for max-heaps and min-heaps:

Max-Heap Order Property: In a max-heap, each parent node is greater than or equal to its child nodes. This ensures that the largest element is always at the root.

Min-Heap Order Property: In a min-heap, each parent node is less than or equal to its child nodes. This ensures that the smallest element is always at the root.

HEAP OPERATIONS

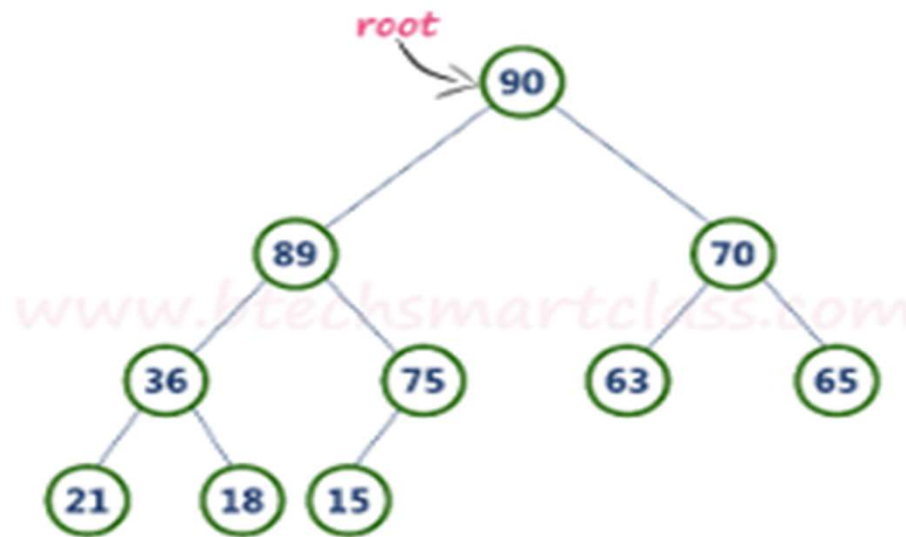
Common heap operations are:

- **Insert:** Adds a new element to the heap while maintaining the heap property.
- **Extract Max/Min:** Removes the maximum or minimum element from the heap and returns it.
- **Heapify:** Converts an arbitrary binary tree into a heap.

MAX HEAP

Max heap is a specialized complete binary tree in which every parent node contains **greater or equal value** than its child nodes.

Example



MAX HEAP

Above tree is satisfying both Ordering property and Structural property according to the Max Heap data structure.

The following operations are performed on a Max heap data structure...

1. Finding Maximum
2. Insertion
3. Deletion

MAX HEAP – FINDING MAXIMUM

Finding Maximum Value Operation in Max Heap:

Finding the node which has maximum value in a max heap is very simple.

In a max heap, the root node has the maximum value than all other nodes.

So, directly we can display root node value as the maximum value in max heap.

MAX HEAP - INSERTION

Insertion Operation in Max Heap

Insertion Operation in max heap is performed as follows.

Step 1 - Insert the new Node as last leaf from left to right.

Step 2 - Compare new Node value with its Parent node.

Step 3 - If new Node value is greater than its parent, then swap both of them.

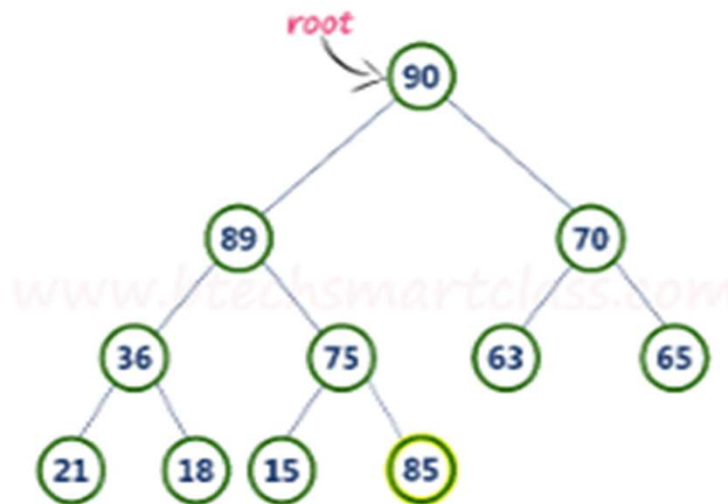
Step 4 - Repeat step 2 and step 3 until new Node value is less than its parent node (or) new Node reaches to root.

MAX HEAP - INSERTION

Example:

Consider the below max heap. Insert a new node with value 85.

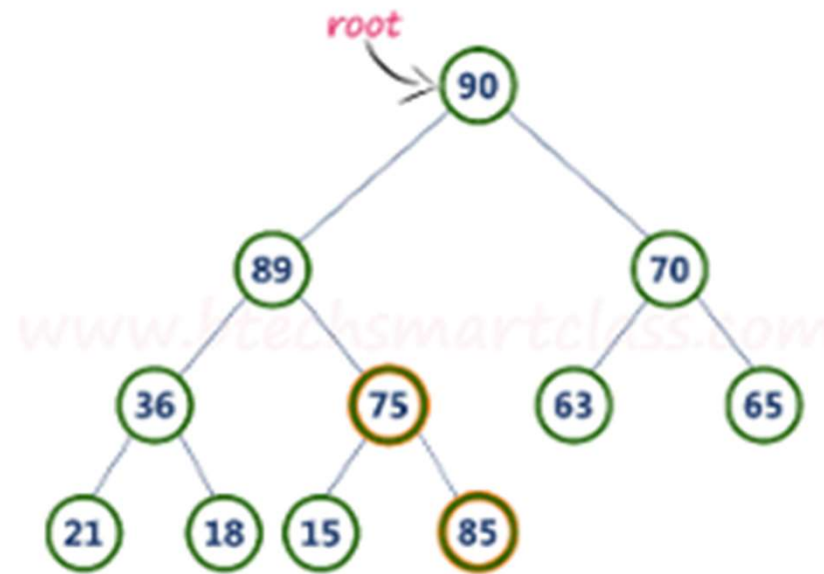
Step 1 - Insert the new Node with value 85 as last leaf from left to right. That means new Node is added as a right child of node with value 75. After adding max heap is as follows.



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

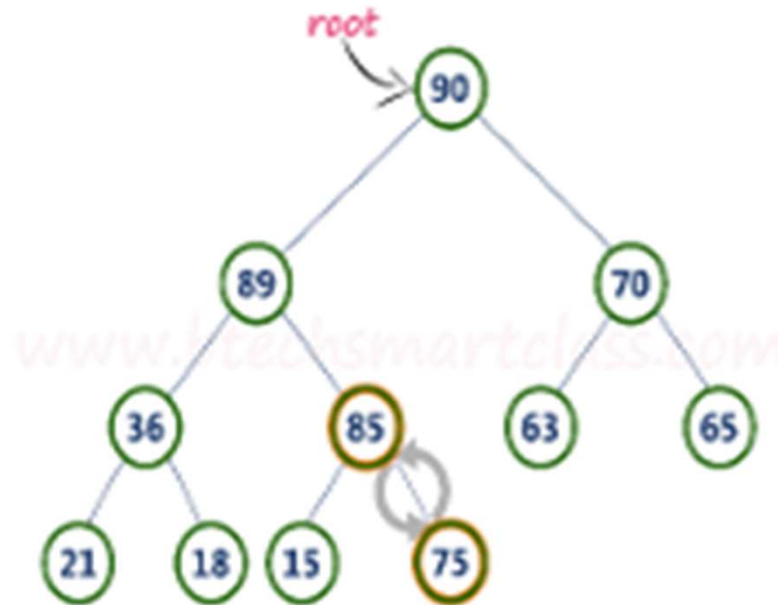
MAX HEAP - INSERTION

Step 2 - Compare new Node value (85) with its Parent node value (75).
That means $85 > 75$



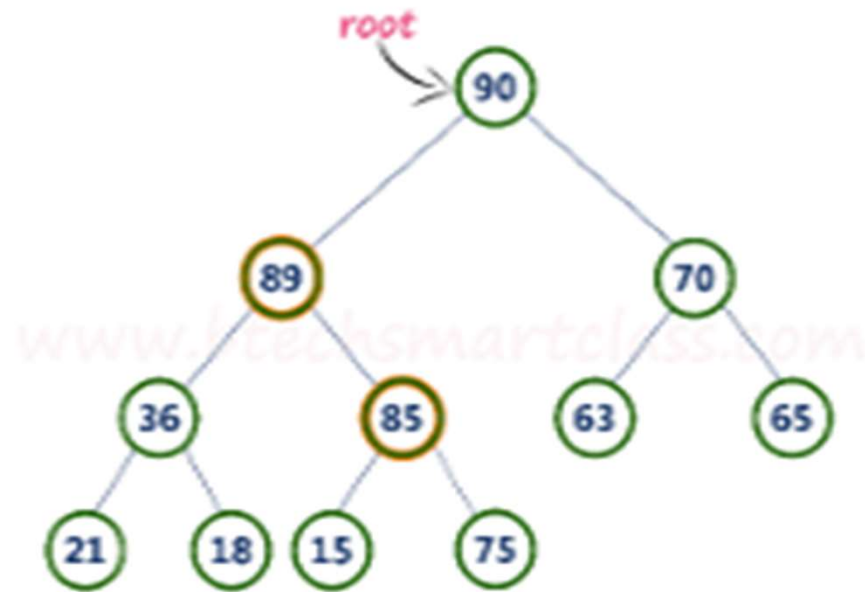
MAX HEAP - INSERTION

Step 3 - Here new Node value (85) is greater than its parent value (75), then swap both of them. After swapping, max heap is as follows.



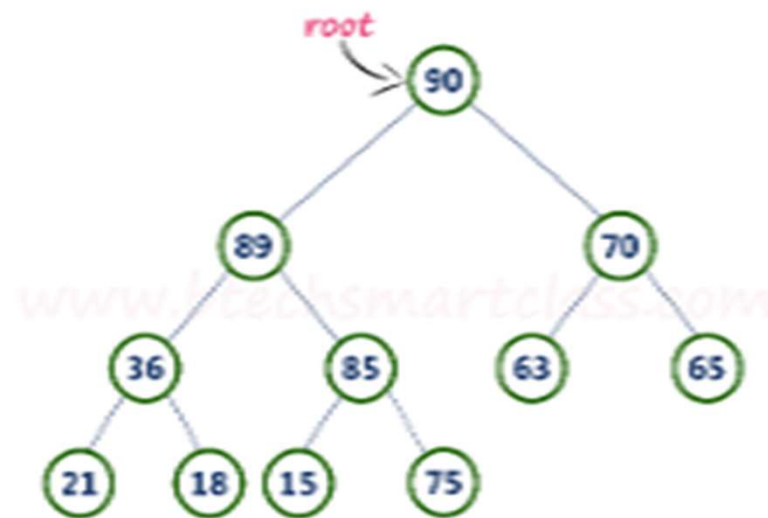
MAX HEAP - INSERTION

Step 4 - Now, again compare newNode value (85) with its parent node value (89).



MAX HEAP - INSERTION

Here, new Node value (85) is smaller than its parent node value (89). So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



MAX HEAP- DELETION

Deletion Operation in Max Heap

In a max heap, deleting the last node is very simple as it does not disturb max heap properties. Deleting root node from a max heap is little difficult as it disturbs the max heap properties. We use the following steps to delete the root node from a max heap.

MAX HEAP- DELETION

Step 1 - Swap the root node with last node in max heap

Step 2 - Delete last node.

Step 3 - Now, compare root value with its left child value.

Step 4 - If root value is smaller than its left child, then compare left child with its right sibling. Else goto Step 6

Step 5 - If left child value is larger than its right sibling, then swap root with left child otherwise swap root with its right child.

Step 6 - If root value is larger than its left child, then compare root value with its right child value.

Step 7 - If root value is smaller than its right child, then swap root with right child otherwise stop the process.

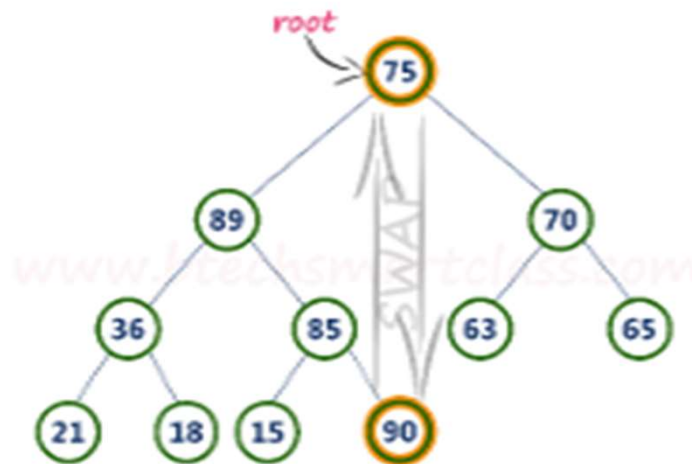
Step 8 - Repeat the same until root node fixes at its exact position.

MAX HEAP- DELETION

Example

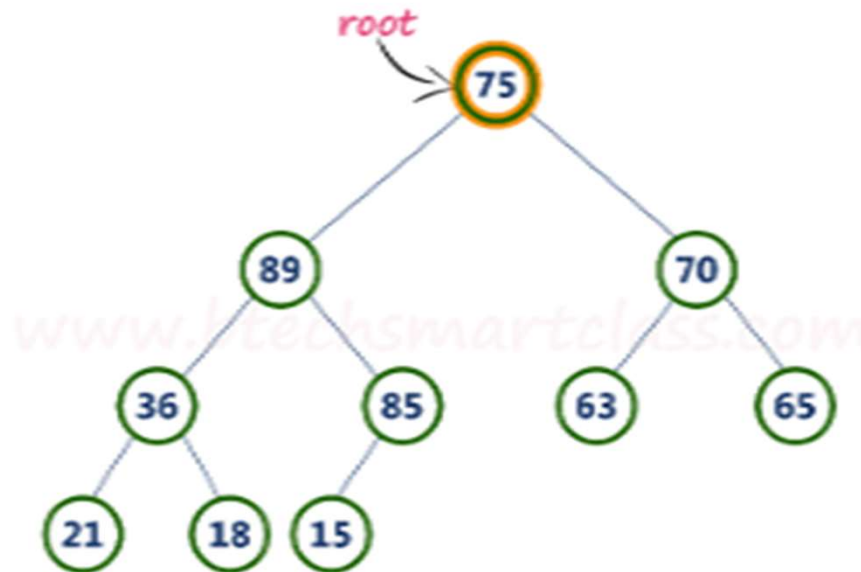
Consider the below max heap. Delete root node (90) from the max heap.

Step 1 - Swap the root node (90) with last node 75 in max heap. After swapping max heap is as follows...



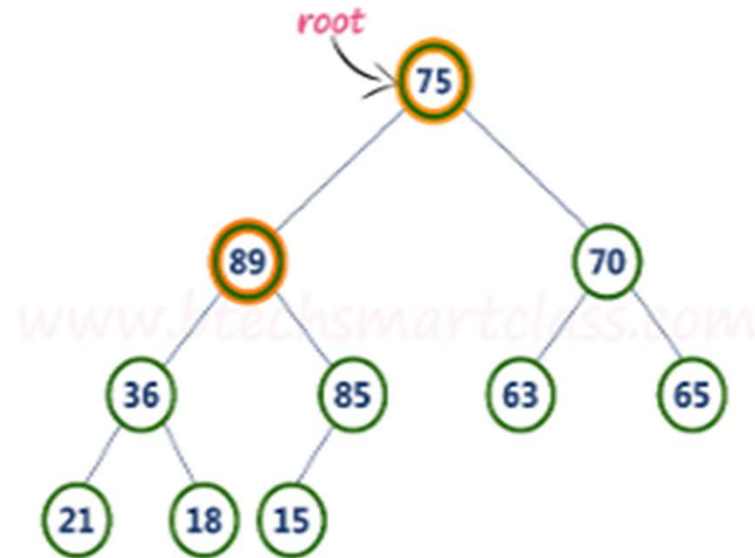
MAX HEAP- DELETION

Step 2 - Delete last node. Here the last node is 90. After deleting node with value 90 from heap, max heap is as follows...



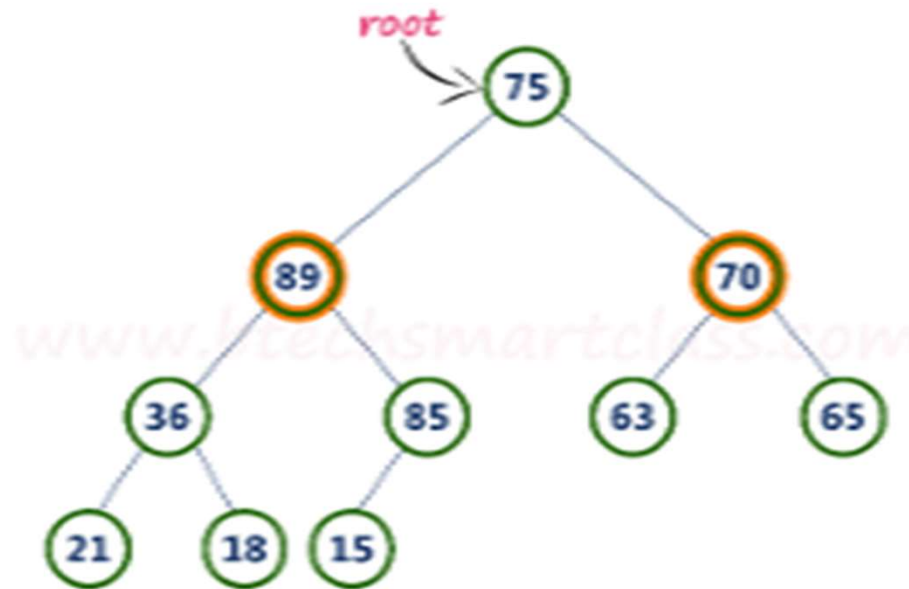
MAX HEAP- DELETION

Step 3 - Compare root node (75) with its left child (89).



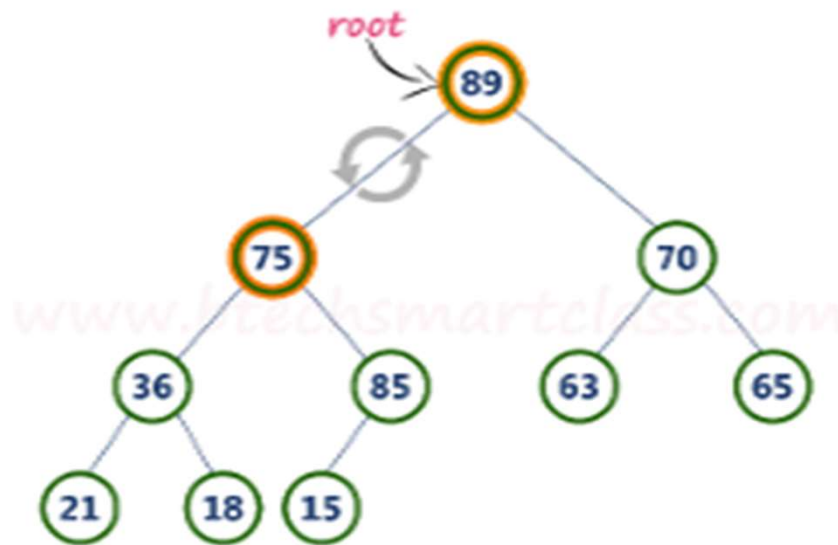
MAX HEAP- DELETION

Here, root value (75) is smaller than its left child value (89). So, compare left child (89) with its right sibling (70).



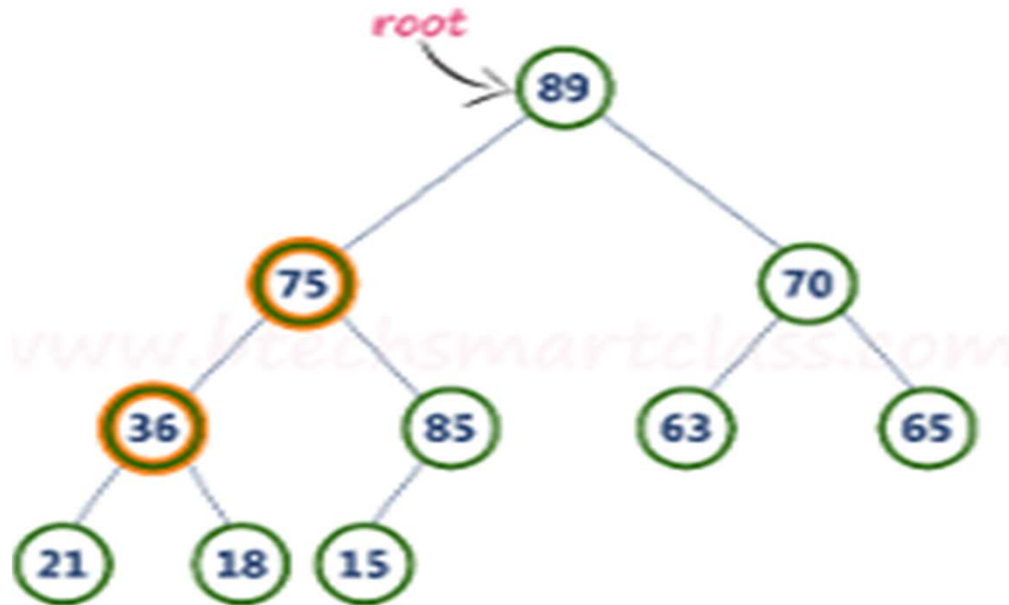
MAX HEAP- DELETION

Step 4 - Here, left child value (89) is larger than its right sibling (70), So, swap root (75) with left child (89).



MAX HEAP- DELETION

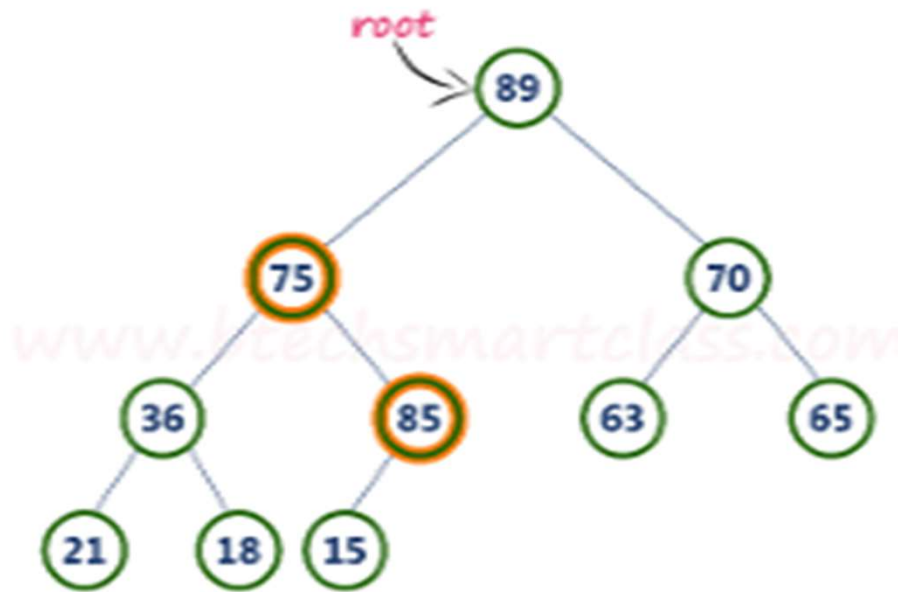
Step 5 - Now, again compare 75 with its left child (36).



Here, node with value 75 is larger than its left child. So, we compare node 75 with its right child 85.

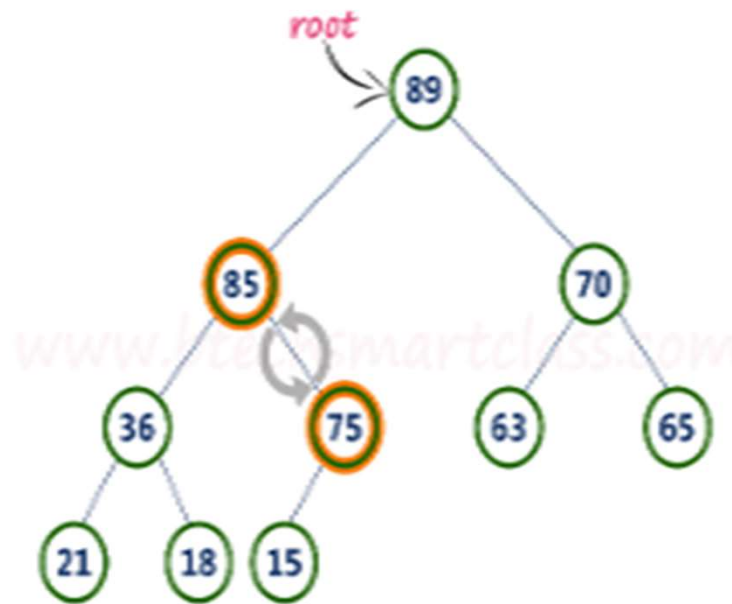
MAX HEAP- DELETION

Here, node with value 75 is larger than its left child. So, we compare node 75 with its right child 85.



MAX HEAP- DELETION

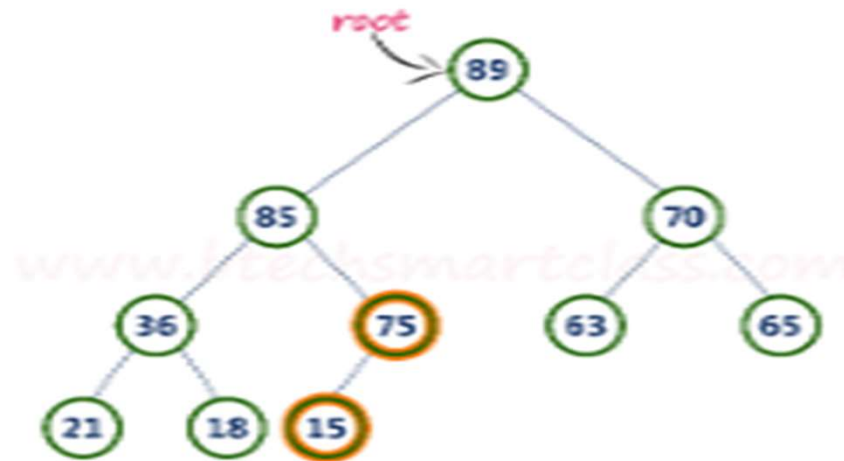
Step 6 - Here, node with value 75 is smaller than its right child (85). So, we swap both of them. After swapping max heap is as follows.



MAX HEAP- DELETION

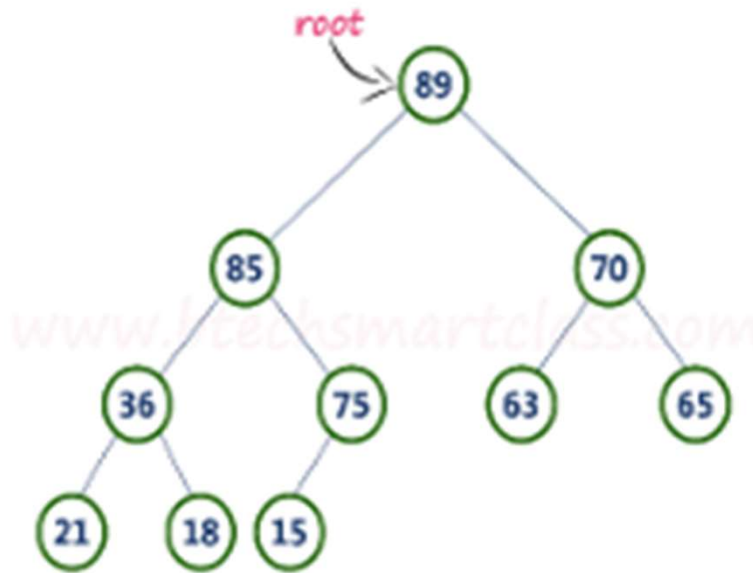
Step 7 - Now, compare node with value 75 with its left child (15).

Here, node with value 75 is larger than its left child (15) and it does not have right child. So we stop the process.



MAX HEAP- DELETION

Finally, max heap after deleting root node (90) is as follows.



ARRAY REPRESENTATION OF HEAPS

Array Representation of Heaps

Heaps can be efficiently stored in arrays because they are complete binary trees. This representation makes it easy to calculate the parent and child indices using simple arithmetic.

Storing Heaps in Arrays

In an array representation of a heap:

The root node is stored at index zero.

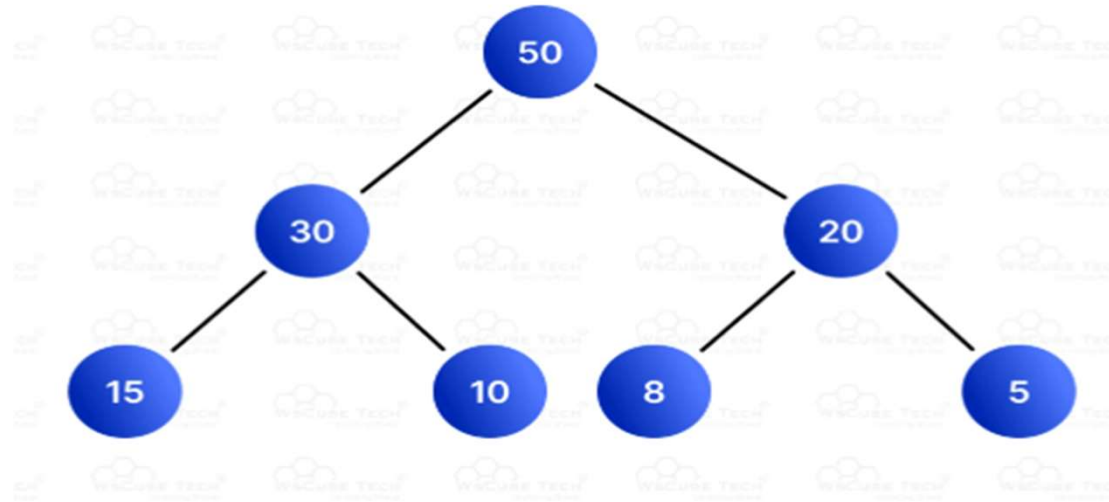
For any node at index i :

- Its left child is at index $2i + 1$.
- Its right child is at index $2i + 2$.

Its parent is at index $(i - 1) // 2$ (if i is not zero).

HEAPIFY

Example: Consider the following max-heap:

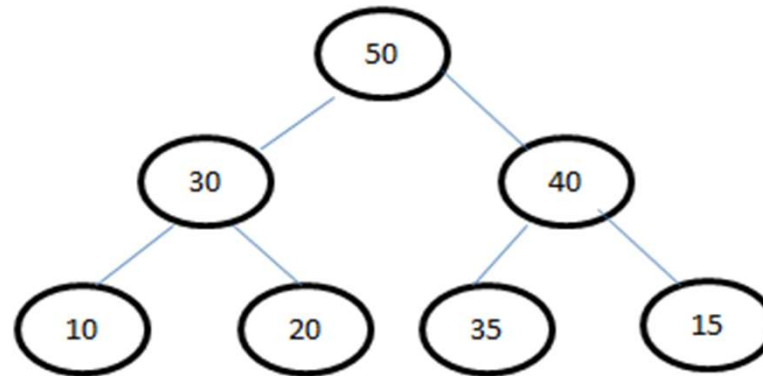


The array representation would be:

0	1	2	3	4	5	6
50	30	20	15	10	8	5

INSERTION IN A MAX HEAP (ARRAY REPRESENTATION)

Suppose the Max Heap is:



Let the input array be

0	1	2	3	4	5	6
50	30	40	10	20	35	15

INSERTION IN A MAX HEAP (ARRAY REPRESENTATION)

Suppose we insert 45.

Current array:

0	1	2	3	4	5	6	7
50	30	40	10	20	35	15	45

Find parent

New node index at 7

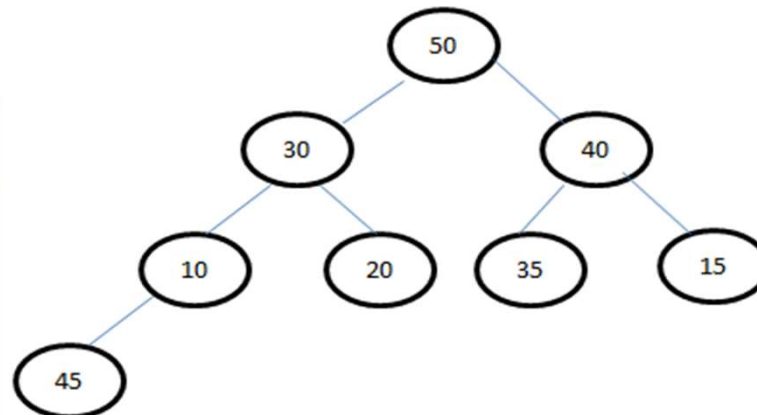
$$\text{Parent Index} = \frac{i - 1}{2}$$

where i is the index of the child.

Parent index = 3

Value = 10

$$\frac{7 - 1}{2} = 3$$



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

INSERTION IN A MAX HEAP (ARRAY REPRESENTATION)

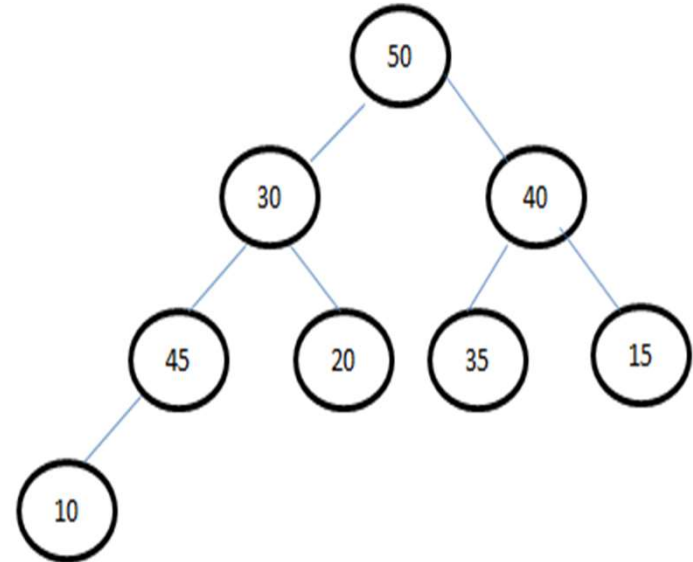
Compare

$45 > 10$

Swap

Array becomes

0	1	2	3	4	5	6	7
50	30	40	45	20	35	15	10



Now the new position of 45 is index 3.

INSERTION IN A MAX HEAP (ARRAY REPRESENTATION)

Find the new parent.

Parent index = 1

Value = 30

Compare

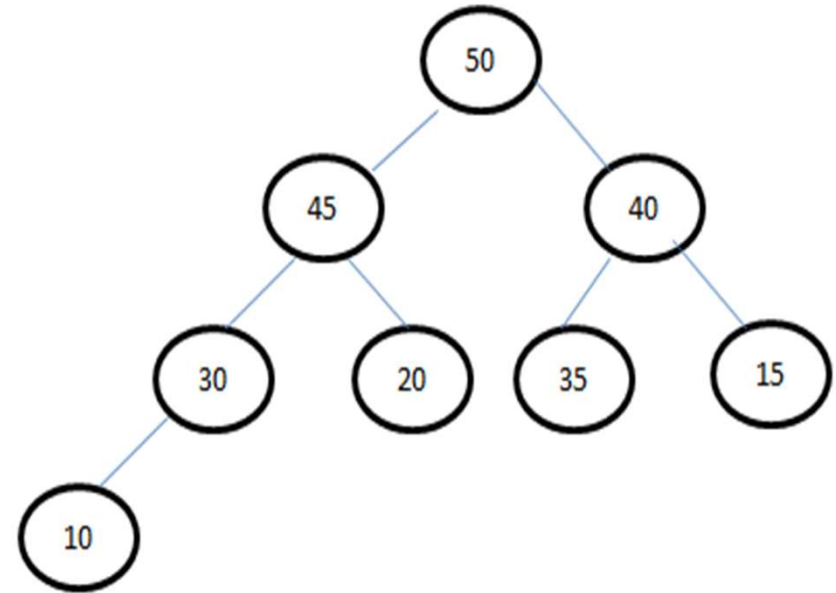
$45 > 30$

Swap

Array becomes

0	1	2	3	4	5	6	7
50	45	40	30	20	35	15	10

Now 45 is at index 1.



INSERTION IN A MAX HEAP (ARRAY REPRESENTATION)

Find parent

$$\frac{1 - 1}{2} = 0$$

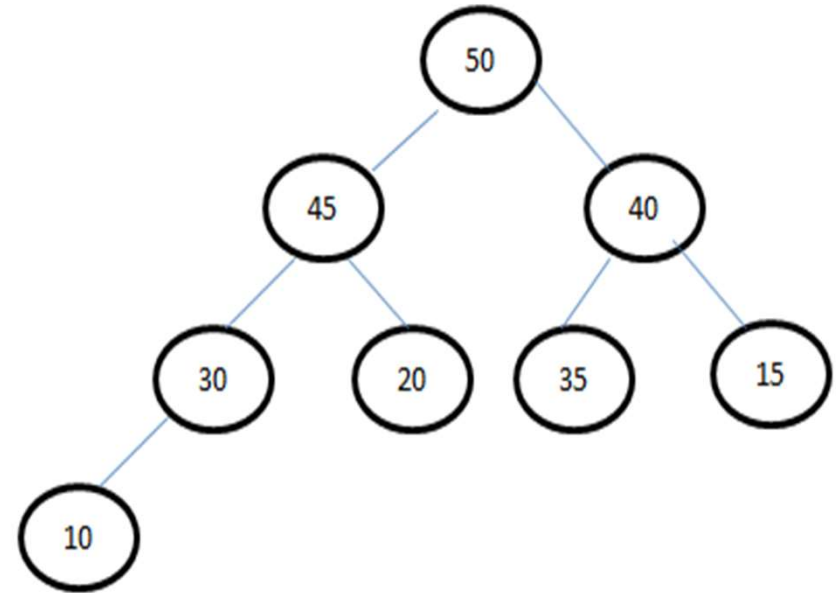
Parent index = 0

Value = 50

Compare

$45 < 50$

Stop.



DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Rule: In a Max Heap, we always delete the root element (the maximum element).

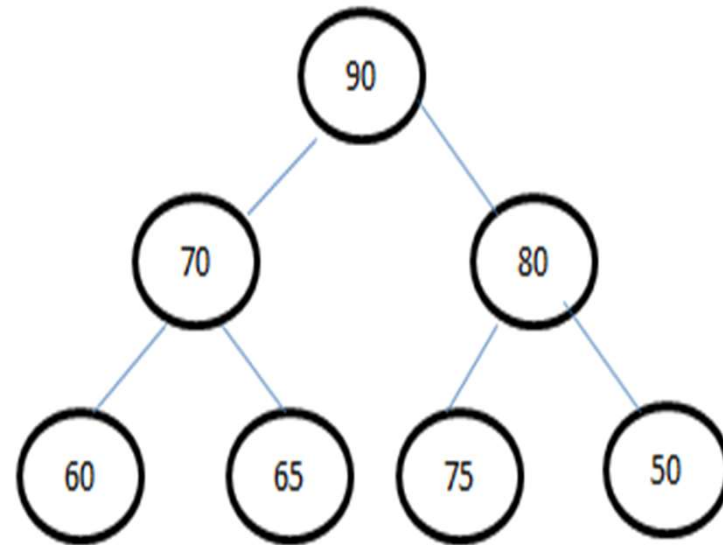
Algorithm

1. Delete the root.
2. Move the last element to the root.
3. Reduce the heap size by 1.
4. Compare the root with its children.
5. Swap with the larger child if the root is smaller.
6. Repeat until the heap property is restored.

This process is called Heapify Down or Percolate Down

DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Initial Max Heap:



Array Representation

0	1	2	3	4	5	6
90	70	80	60	65	75	50

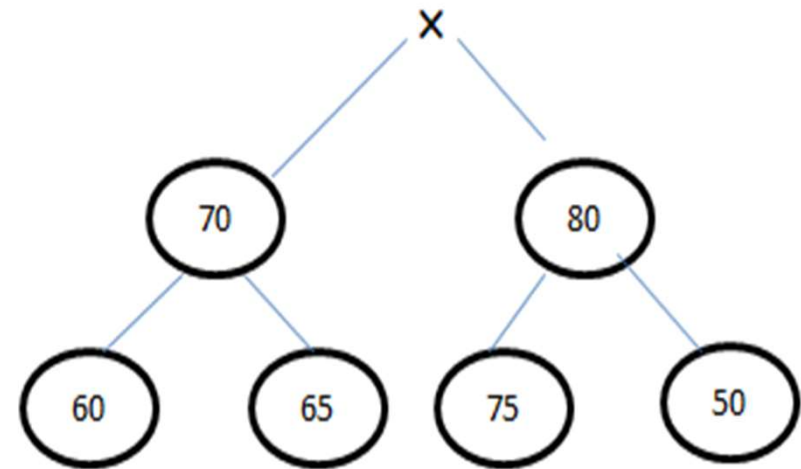
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Delete the Root

Delete 90.

Now the heap looks like this:



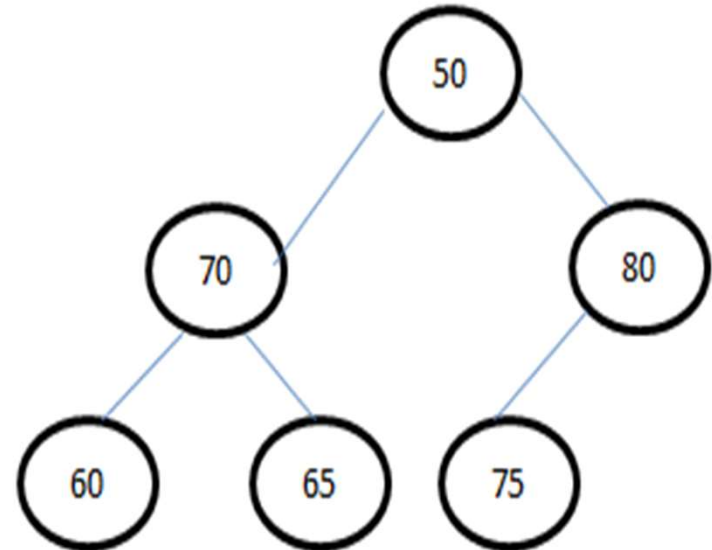
DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Move the Last Element to the Root

Last element = 50

Move it to the root.

0	1	2	3	4	5	6
50	70	80	60	65	75	90



DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Compare with Children

Current node = 50

Find the children using the formulas:

Parent index $= i = 0$

Left child Index : $2i+1 = 2(0)+1 = 1$

Right child index: $2i + 2 = 2(0)+2 = 2$

Children:

- Left = 70
- Right = 80

Choose the larger child.

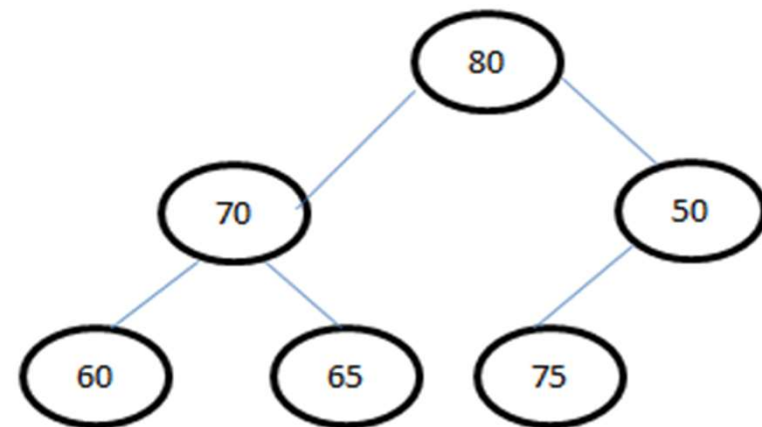
80 is larger.

Since

$50 < 80$

Swap them.

0	1	2	3	4	5
80	70	50	60	65	75



0	1	2	3	4	5
50	70	80	60	65	75

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Continue Heapify

Current position of 50 is index 2.

Find the children using the formulas:

Parent index = $i = 2$

Left child Index : $2i+1 = 2(2)+1 = 5$

Right child index: $2i + 2 = 2(2)+2 = 6$

Index 5 = 75

Index 6 = Not present

Compare

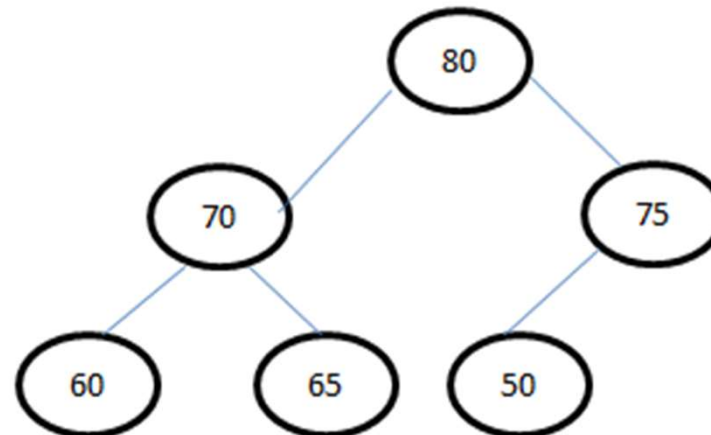
$50 < 75$

Swap.

0	1	2	3	4	5
80	70	50	60	65	75

Array representation:

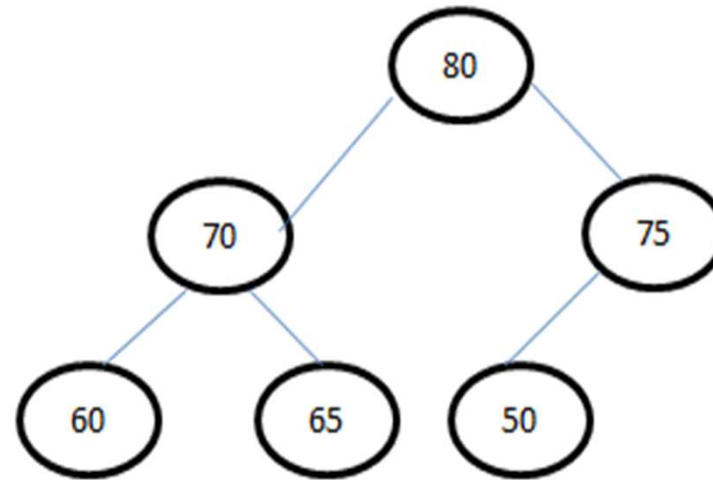
0	1	2	3	4	5
80	70	75	60	65	50



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

DELETION IN A MAX HEAP (ARRAY REPRESENTATION)

Final Heap



Final Array

0	1	2	3	4	5
80	70	75	60	65	50

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

HEAP DATA STRUCTURE APPLICATIONS

Heaps have various applications, like:

- Heaps are commonly used to implement priority queues, where elements are retrieved based on their priority (maximum or minimum value).
- Heapsort is a sorting algorithm that uses a heap to sort an array in ascending or descending order.
- Heaps are used in graph algorithms like Dijkstra's algorithm and Prim's algorithm for finding the shortest paths and minimum spanning trees.

ADVANTAGES OF HEAP TREE IN DATA STRUCTURE

1. Efficient priority queue operations ($O(\log n)$ for insertion and deletion)
2. Memory-efficient with contiguous storage
3. Suitable for heap sort ($O(n \log n)$ time complexity)
4. Effective for dynamic datasets
5. Supports quick access to minimum/maximum elements

DISADVANTAGES OF HEAP TREE IN DATA STRUCTURE

1. Inefficient search operations ($O(n)$ time complexity)
2. Elements are not stored in a sorted order
3. More complex implementation and maintenance
4. Fixed structure can lead to inefficient memory use
5. Balancing overhead during heapify operations

QUESTIONS

1. Construct a Min Heap from the following sequence of elements: 35, 33, 42, 10, 14, 19, 27, 44, 26, 31. Show the heap after each insertion and explain the heap property that is maintained at each step. Then, perform a delete operation to remove the minimum element and restore the heap.
2. Starting with an empty Max Heap, insert the following elements: 5, 12, 7, 9, 15, 20, 3, 17. After each insertion, demonstrate how the Max Heap property is preserved. Then, perform a delete operation to remove the maximum element and restore the heap.
3. Differentiate between min-heaps and max-heaps

Short Questions:

1. List two key applications of Min Heaps in real-world scenarios.
2. What is a priority queue, and why are heaps suitable for implementing it?

GRAPHS

Graphs – Terminology, Representations, Basic Search and Traversals, Connected Components and Biconnected Components, applications

GRAPHS

Definition:

Graph is a **non-linear data structure**. It contains a set of points known as nodes (or vertices) and a set of links known as edges (or Arcs). Here edges are used to connect the vertices.

A graph is defined as follows.

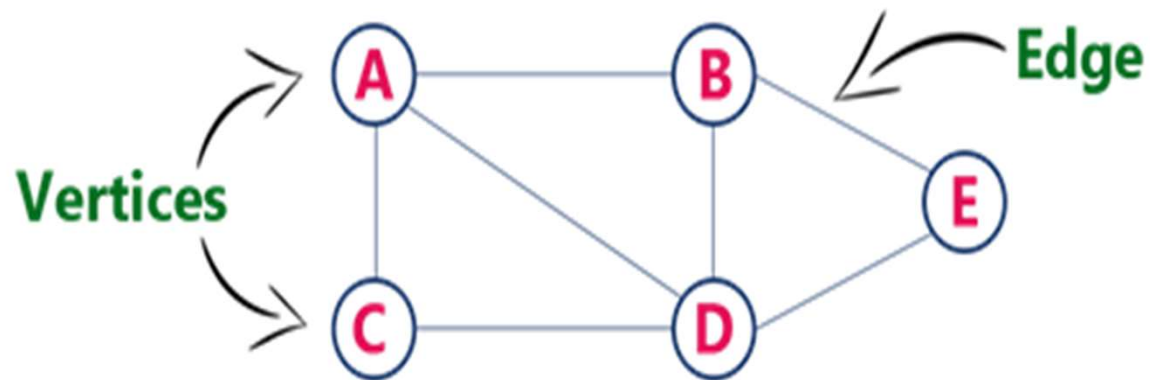
- Graph is a collection of **vertices and arcs** in which vertices are connected with arcs
- Graph is a collection **of nodes and edges** in which nodes are connected with edges

Generally, a graph G is represented as $G = (V , E)$, where V is set of **vertices** and E is **set of edges**.

GRAPHS

Example :

The following is a graph with 5 vertices and 6 edges. This graph G can be defined as $G = (V, E)$ Where $V = \{A, B, C, D, E\}$ and $E = \{(A, B), (A, C), (A, D), (B, D), (C, D), (B, E), (E, D)\}$.



Graph Terminology:

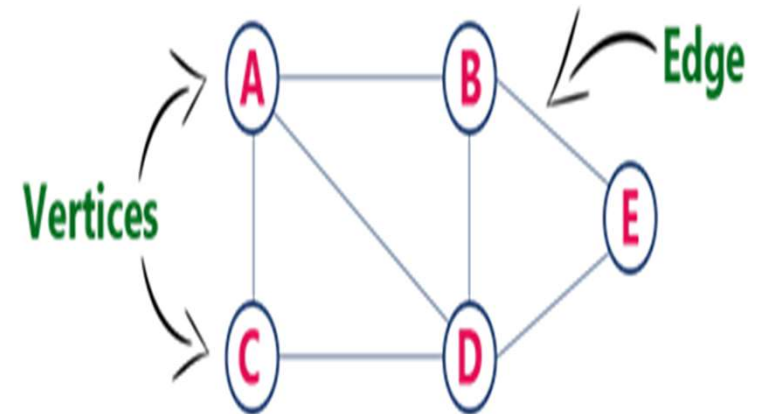
We use the following terms in graph data structure.

Vertex:

Individual data element of a graph is called as Vertex. Vertex is also known as node. In above example graph, A, B, C, D & E are known as vertices.

Edge:

An edge is a connecting link between two vertices. Edge is also known as Arc. An edge is represented as (startingVertex, endingVertex). For example, in above graph the link between vertices A and B is represented as (A,B). In above example graph, there are 7 edges (i.e., (A,B), (A,C), (A,D), (B,D), (B,E), (C,D), (D,E)).



GRAPHS

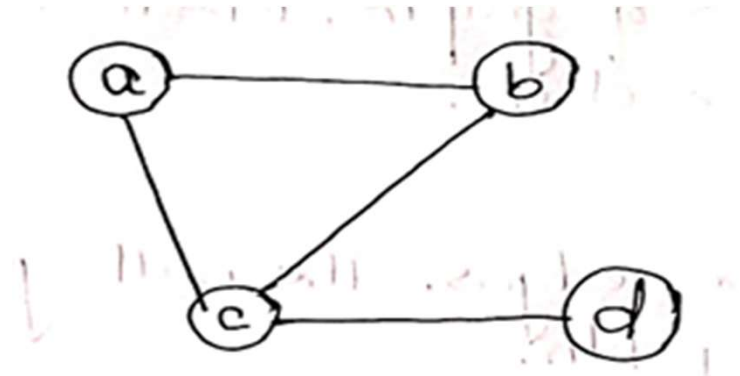
Edges are three types.

- 1. Undirected Edge** - An undirected edge is a bidirectional edge. If there is undirected edge between vertices A and B then edge (A , B) is equal to edge (B , A).
- 2. Directed Edge** - A directed edge is a unidirectional edge. If there is directed edge between vertices A and B then edge (A , B) is not equal to edge (B , A).
- 3. Weighted Edge** - A weighted edge is a edge with value (cost) on it.

Degree of Vertex

The degree of vertex V of a graph G (denoted by $\deg(v)$) is the number of edges incident with the vertex v

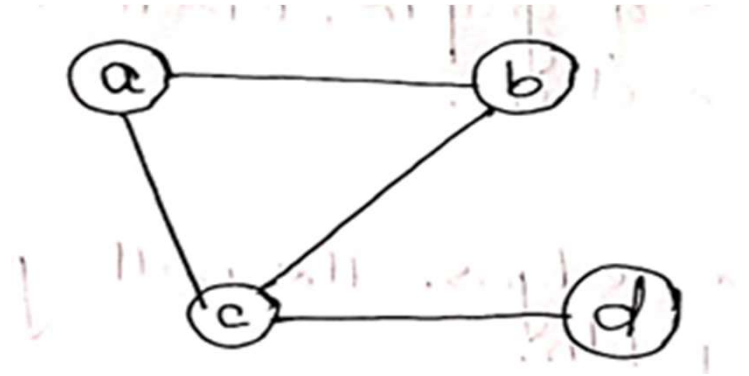
Vertex	Degree
a	2
b	2
c	3
d	1



Even or Odd Vertex:

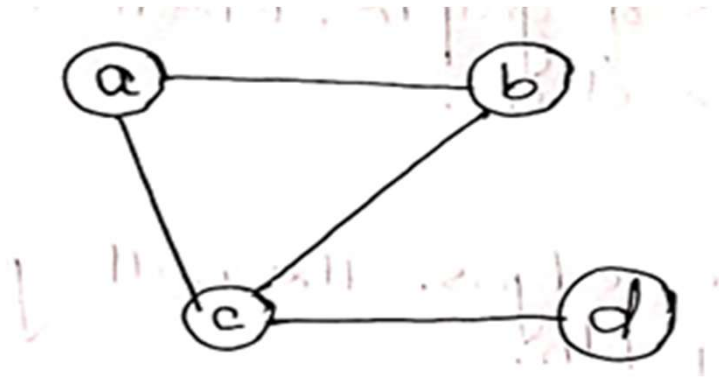
If the degree of a vertex is even, the vertex is called an even vertex and If the degree of a vertex is odd, the vertex is called an odd vertex.

Vertex	Degree	Even or Odd vertex
a	2	Even
b	2	Even
c	3	Odd
d	1	Odd



Degree of a Graph:

- The degree of a graph is the largest vertex degree of that graph.

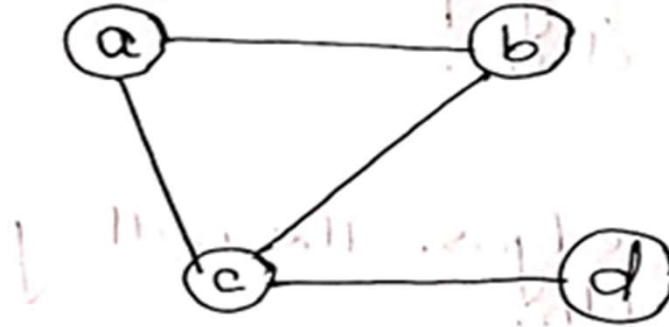


- For the above graph the Degree of the graph is 3

GRAPHS

The Handshaking Lemma:

In a graph, the sum of all the degrees of all the vertices is equal to twice the number of edges



The handshaking lemma = $2+2+3+1 = 8$

Twice the number of edges: $4 \times 2 = 8$

TYPES OF GRAPHS:

➤ There are different types of graphs:

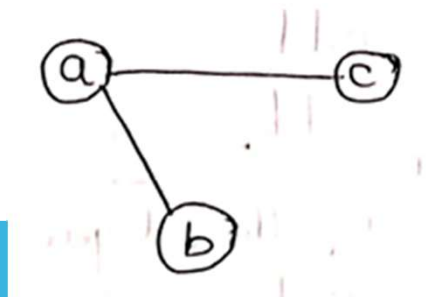
1. Null Graph:

A Graph has no edges is called Null Graph.



2. Simple Graph:

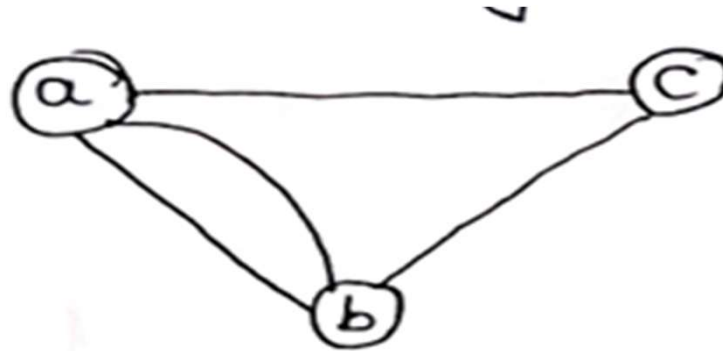
A Graph is called simple graph or strict graph if the graph is undirected and does not contain any loops or multiple edges.



TYPES OF GRAPHS:

3. Multi Graph:

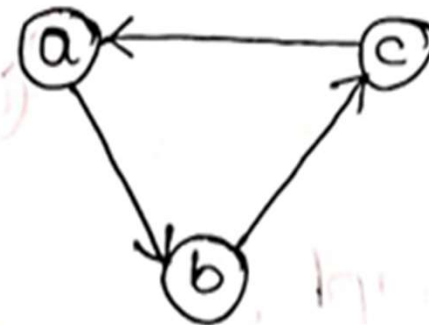
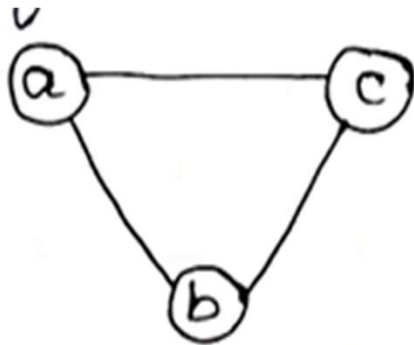
If in a graph multiple edges are there in between same set of vertices are called Multi Graph. In other words, it is graph having at-least one loop.



TYPES OF GRAPHS:

4. Directed and Undirected Graph:

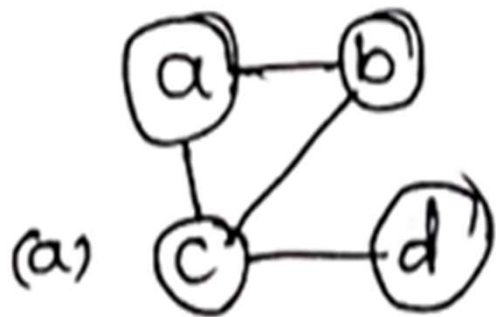
A Graph $G = (V, E)$ is called a directed graph, if the edge set is made of ordered vertex pair and a graph is called undirected graph, if the edge set is made of unordered vertex pair



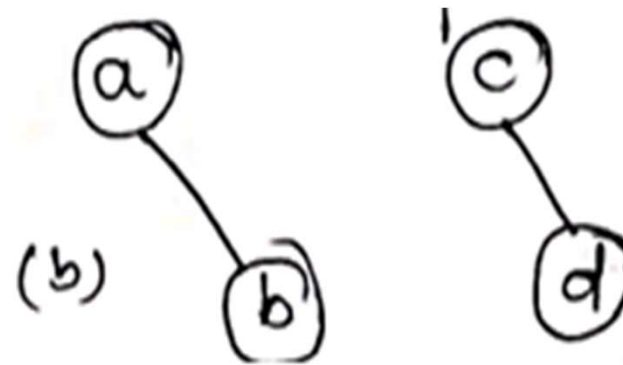
TYPES OF GRAPHS:

5. Connected and Disconnected Graph:

A Graph is connected if any two vertices of the graph are connected by a path; while a graph is disconnected if at-least two vertices of the graph are not connected by a path.



Connected component

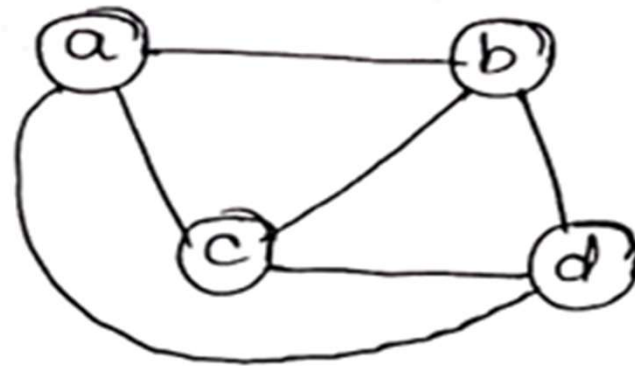


Dis-Connected component

TYPES OF GRAPHS

6. Regular Graph:

A Graph is regular if all the vertices of the graph have the same degree.

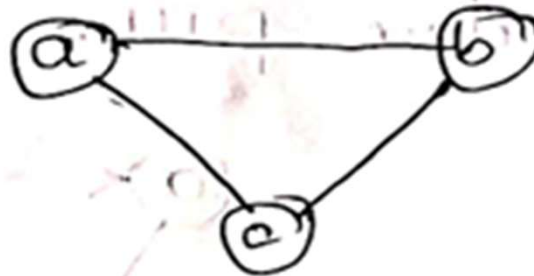


In the above graph degree of each vertex is 3

TYPES OF GRAPHS

7. Complete Graph:

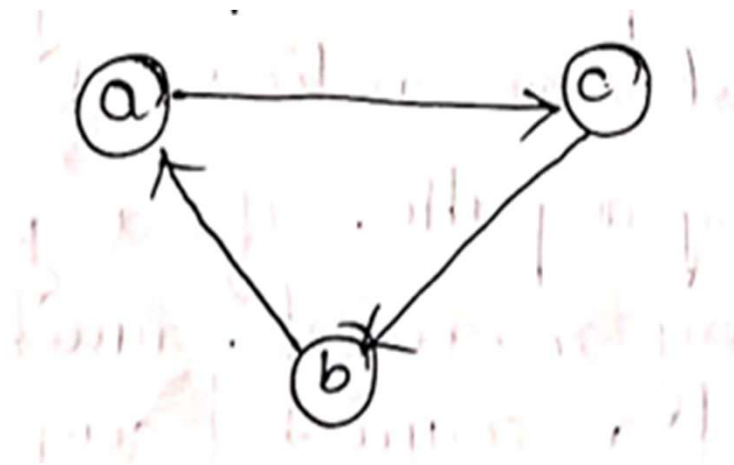
A Graph is called complete graph if every two vertices pair is joined by exactly one edge. The complete graph with n vertices is denoted by K_n



TYPES OF GRAPHS

8. Cyclic Graph:

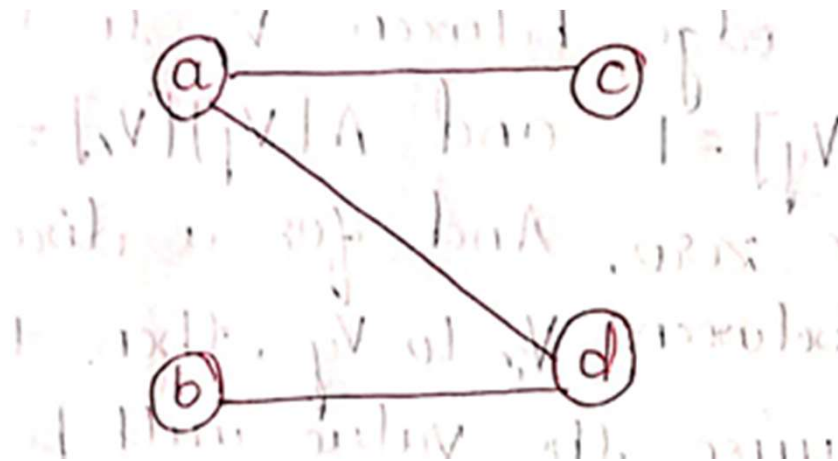
If a graph consists of a single cycle, it is called cyclic graph. The cyclic graph of n vertices is denoted by C_n



TYPES OF GRAPHS

9. Bipartite Graph:

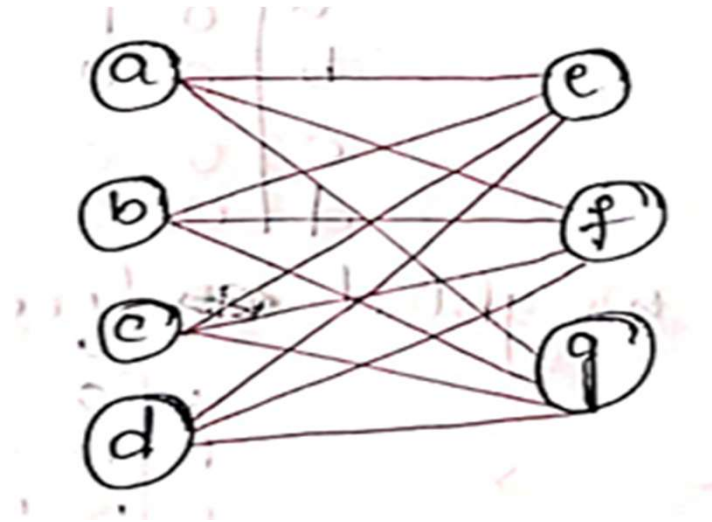
If the vertex-set of a graph G can be split into two disjoint sets, V_1 and V_2 , in such that each edge in the graph joins a vertex in V_1 to a vertex in V_2 , and there are no edges in G that connect two vertices in V_1 or two vertices in V_2 , then the graph G is called a bipartite graph.



TYPES OF GRAPHS

10. Complete Bipartite Graph:

A Complete Bipartite graph is a bipartite graph in which each vertex in the first set is joined to every single vertex in the second set. The complete binary bipartite graph is denoted by K_{xy} , where the graph G contains x vertices in the first set and y vertices in the second set.



GRAPH REPRESENTATIONS

Graph data structure is represented using following representations.

1. Adjacency Matrix
2. Incidence Matrix
3. Adjacency List

GRAPH REPRESENTATIONS

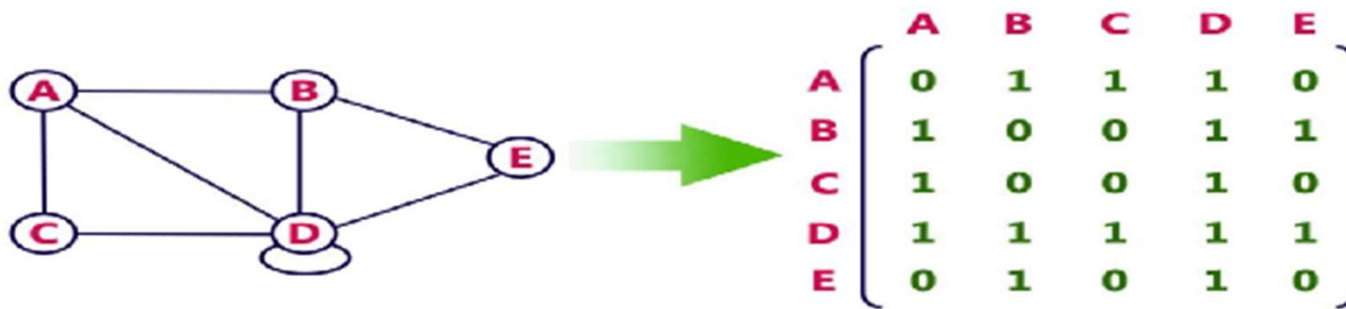
Adjacency Matrix:

In this representation, the graph is represented using a matrix of size total number of vertices by a total number of vertices. That means a graph with 4 vertices is represented using a matrix of size 4X4. In this matrix, both rows and columns represent vertices. This matrix is filled with either 1 or 0. Here, 1 represents that there is an edge from row vertex to column vertex and 0 represents that there is no edge from row vertex to column vertex.

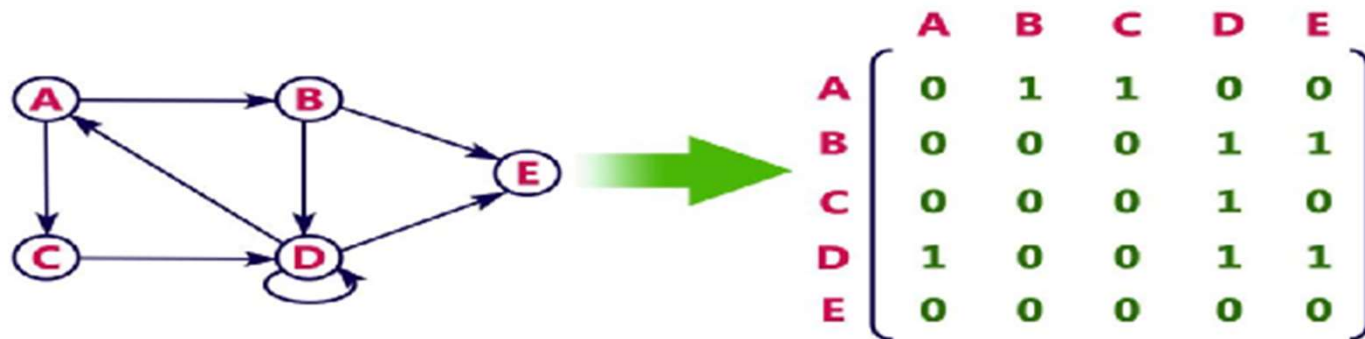
GRAPH REPRESENTATIONS

Adjacency Matrix:

For example, consider the following undirected graph representation.



Directed graph representation



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH REPRESENTATIONS

Incidence Matrix

In this representation, the graph is represented using a matrix of size total number of vertices by a total number of edges. That means graph with 4 vertices and 6 edges is represented using a matrix of size 4×6 .

In this matrix, rows represent vertices and columns represents edges. This matrix is filled with 0 or 1 or -1. Here, 0 represents that the row edge is not connected to column vertex, 1 represents that the row edge is connected as the outgoing edge to column vertex and -1 represents that the row edge is connected as the incoming edge to column vertex.

GRAPH REPRESENTATIONS

For example, consider the following directed graph representation

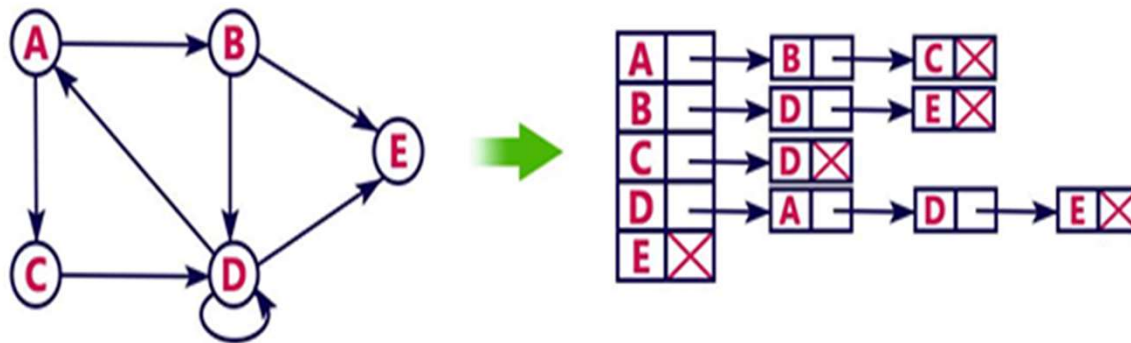


PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH REPRESENTATIONS

Adjacency List

In this representation, every vertex of a graph contains list of its adjacent vertices. For example, consider the following directed graph representation implemented using linked list.



GRAPH TRAVERSAL - DFS

Graph traversal is a technique used for searching a vertex in a graph. The graph traversal is also used to decide the order of vertices visited in the search process. A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of the graph without getting into a looping path.

There are two graph traversal techniques and they are as follows.

1. DFS (Depth First Search)
2. BFS (Breadth First Search)

GRAPH TRAVERSAL - DFS

DFS (Depth First Search):

DFS traversal of a graph produces a **spanning tree** as final result. Spanning Tree is a graph without loops. We use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal.

GRAPH TRAVERSAL - DFS

We use the following steps to implement DFS traversal...

Step 1 - Define a Stack of size total number of vertices in the graph.

Step 2 - Select any vertex as starting point for traversal. Visit that vertex and push it on to the Stack.

Step 3 - Visit any one of the non-visited adjacent vertices of a vertex which is at the top of stack and push it on to the stack.

Step 4 - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.

Step 5 - When there is no new vertex to visit then use back tracking and pop one vertex from the stack.

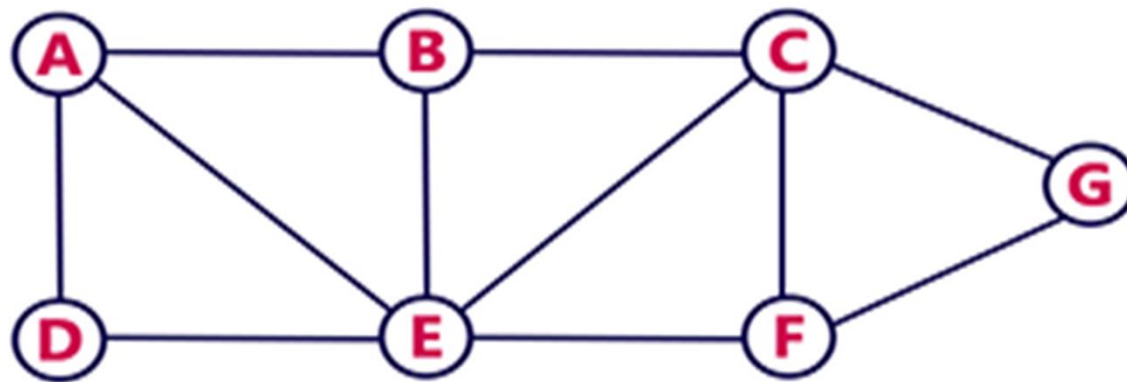
Step 6 - Repeat steps 3, 4 and 5 until stack becomes Empty.

Step 7 - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

GRAPH TRAVERSAL - DFS

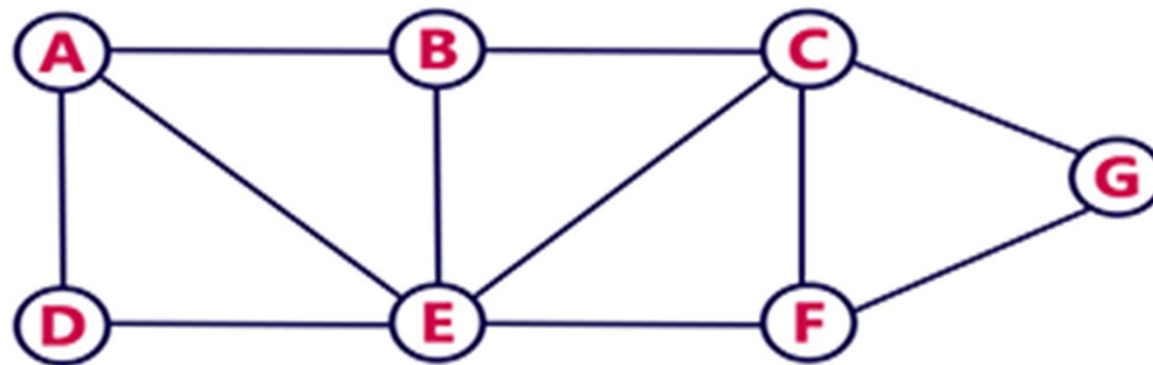
Example

Consider the following example graph to perform DFS traversal



EXAMPLE

Consider the following example graph to perform DFS traversal

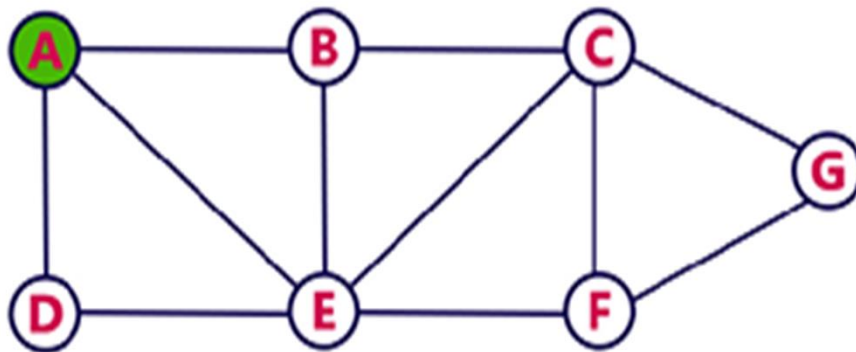


PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH TRAVERSAL - DFS

Step 1:

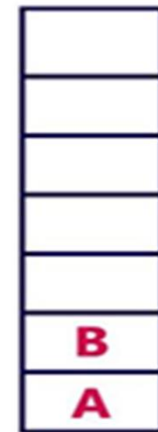
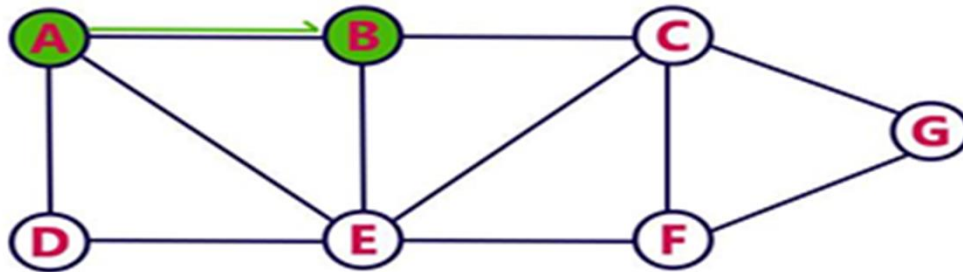
- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



GRAPH TRAVERSAL - DFS

Step 2:

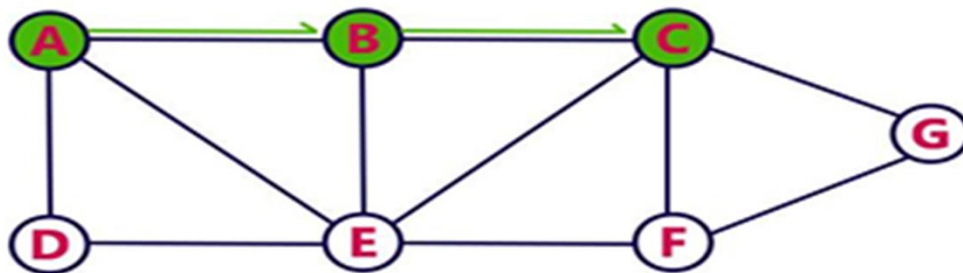
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.



Stack

Step 3:

- Visit any adjacent vertex of **B** which is not visited (**C**).
- Push C on to the Stack.

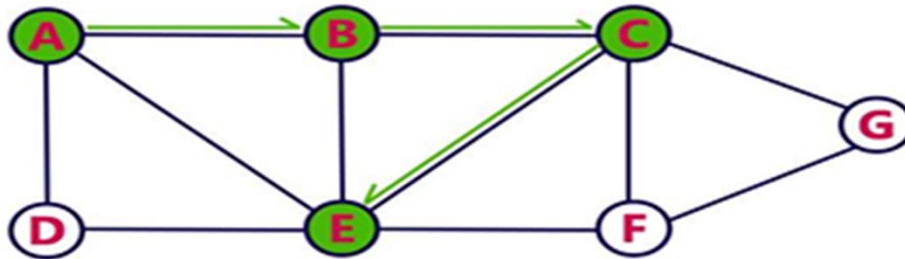


Stack

GRAPH TRAVERSAL - DFS

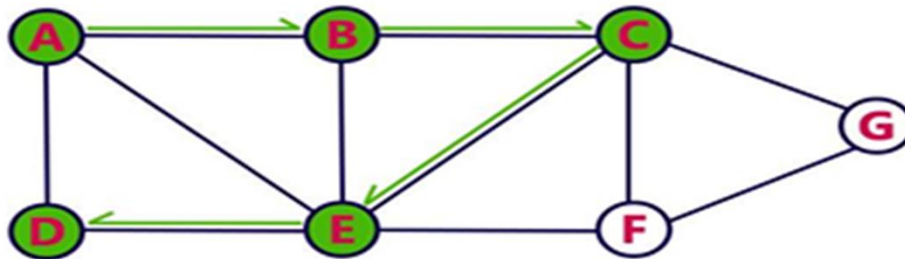
Step 4:

- Visit any adjacent vertex of **C** which is not visited (**E**).
- Push E on to the Stack



Step 5:

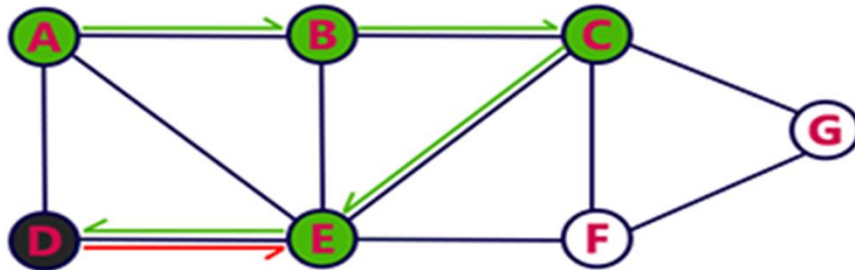
- Visit any adjacent vertex of **E** which is not visited (**D**).
- Push D on to the Stack



GRAPH TRAVERSAL - DFS

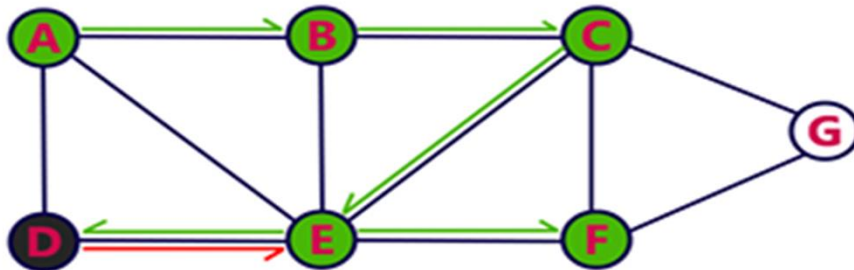
Step 6:

- There is no new vertex to be visited from D. So use back track.
- Pop D from the Stack.



Step 7:

- Visit any adjacent vertex of E which is not visited (F).
- Push F on to the Stack.



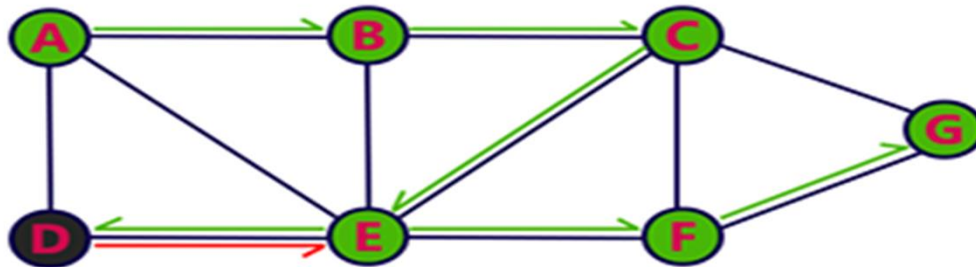
PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH TRAVERSAL - DFS

Step 8:

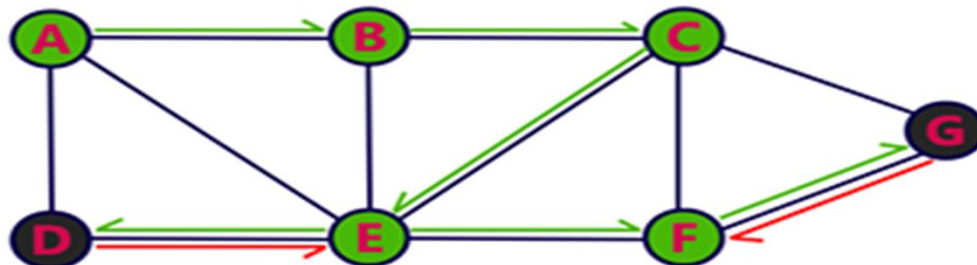
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.



Stack

Step 9:

- There is no new vertex to be visited from G. So use back track.
- Pop G from the Stack.

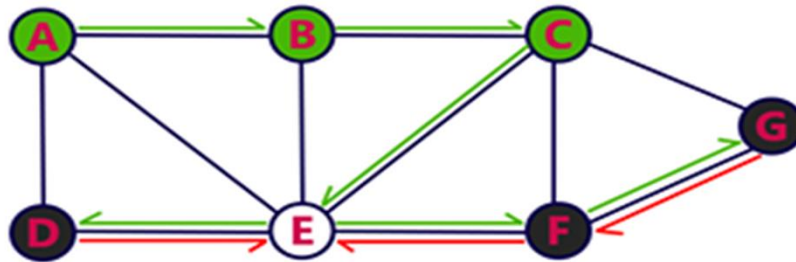


Stack

GRAPH TRAVERSAL - DFS

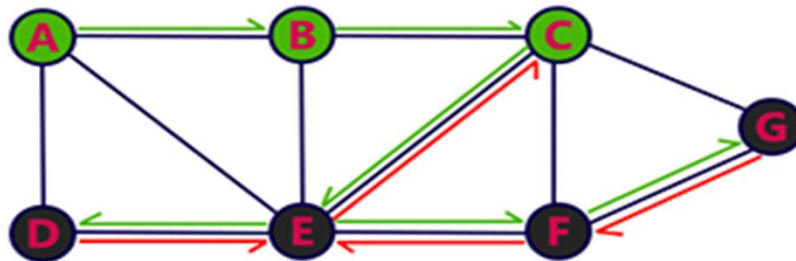
Step 10:

- There is no new vertex to be visited from F. So use back track.
- Pop F from the Stack.



Step 11:

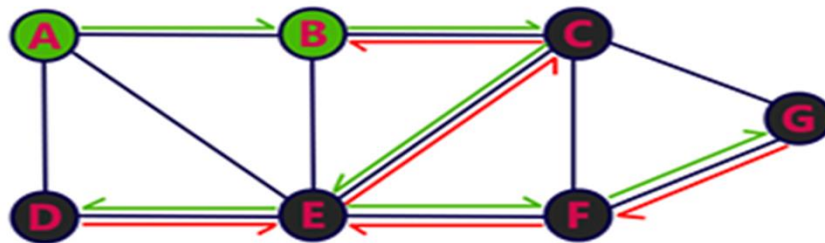
- There is no new vertex to be visited from E. So use back track.
- Pop E from the Stack.



GRAPH TRAVERSAL - DFS

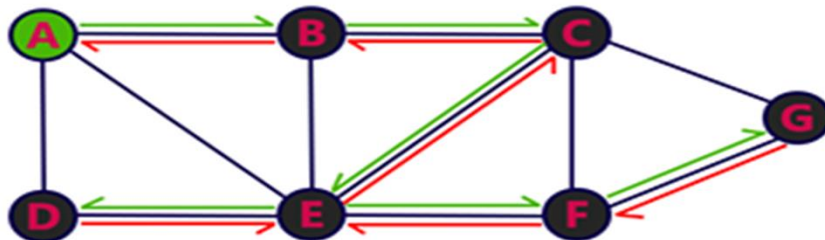
Step 12:

- There is no new vertex to be visited from C. So use back track.
- Pop C from the Stack.



Step 13:

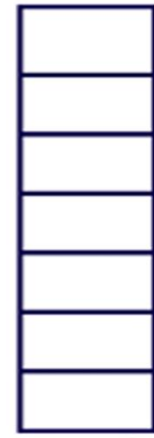
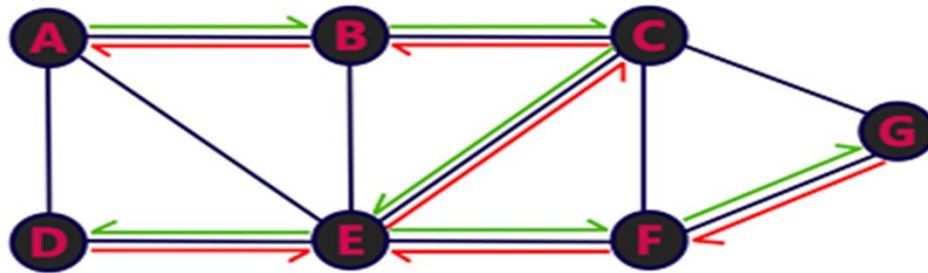
- There is no new vertex to be visited from B. So use back track.
- Pop B from the Stack.



GRAPH TRAVERSAL - DFS

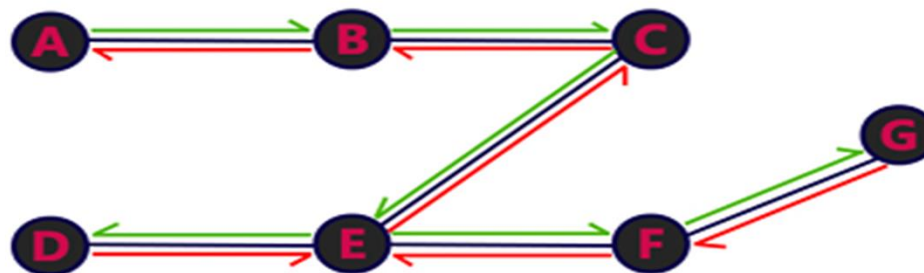
Step 14:

- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.



Stack

- Stack became Empty. So stop DFS Traversal.
- Final result of DFS traversal is following spanning tree.



BFS (BREADTH FIRST SEARCH)

BFS traversal of a graph produces a **spanning tree** as final result. Spanning Tree is a graph without loops. We use **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal.

GRAPH TRAVERSAL - BFS

Step 1 - Define a Queue of size total number of vertices in the graph.

Step 2 - Select any vertex as starting point for traversal. Visit that vertex and insert it into the Queue.

Step 3 - Visit all the non-visited adjacent vertices of the vertex which is at front of the Queue and insert them into the Queue.

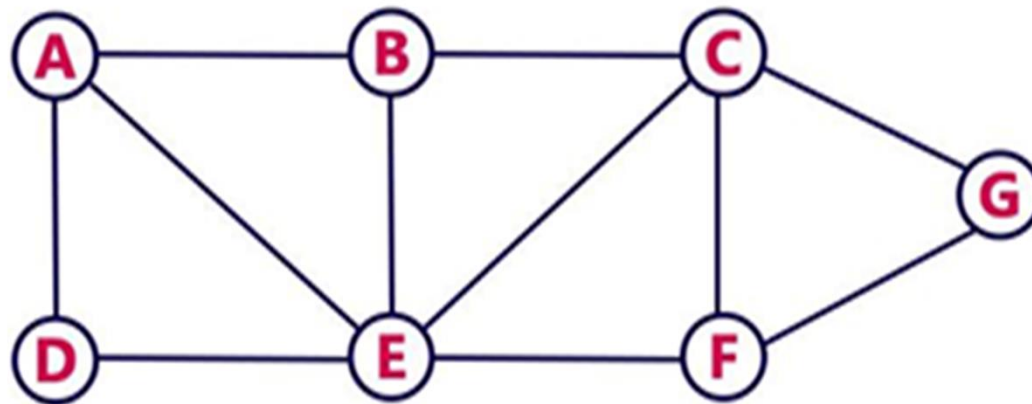
Step 4 - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.

Step 5 - Repeat steps 3 and 4 until queue becomes empty.

Step 6 - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

EXAMPLE

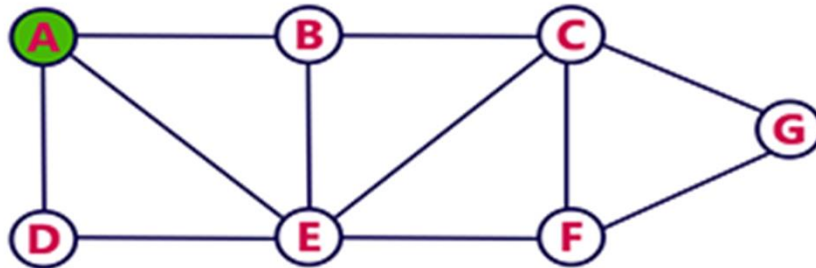
Consider the following example graph to perform BFS traversal



GRAPH TRAVERSAL - BFS

Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

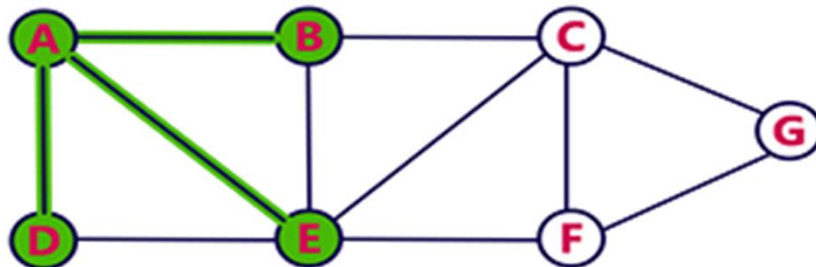


Queue



Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D, E, B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..



Queue

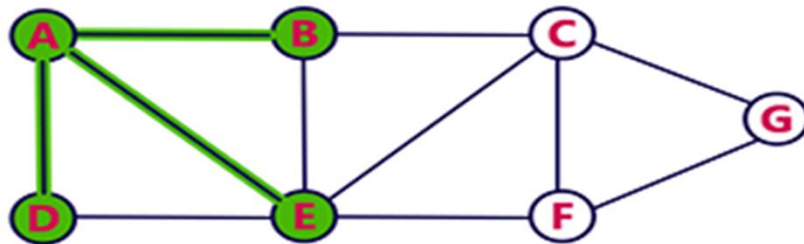


PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH TRAVERSAL - BFS

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

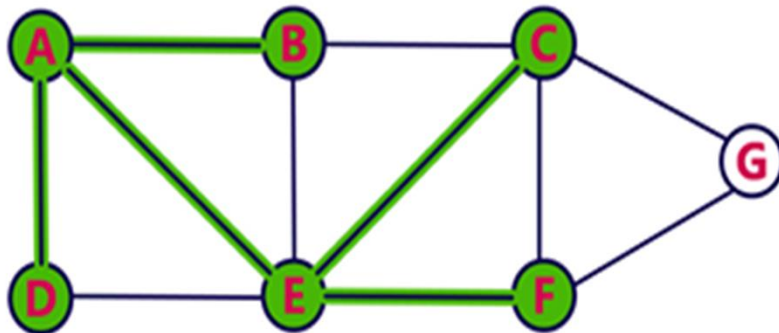


Queue



Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C, F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.



Queue

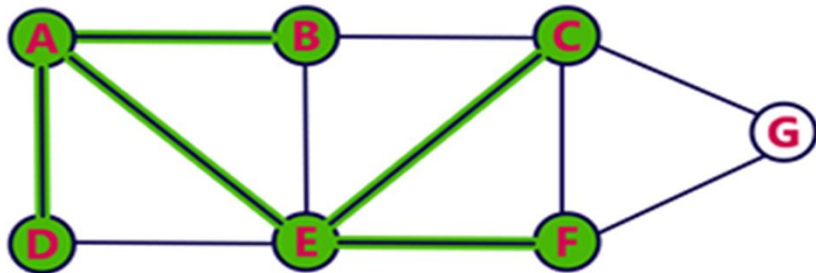


PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH TRAVERSAL - BFS

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

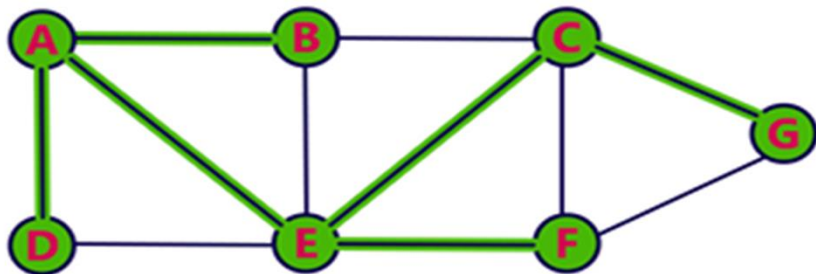


Queue



Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



Queue

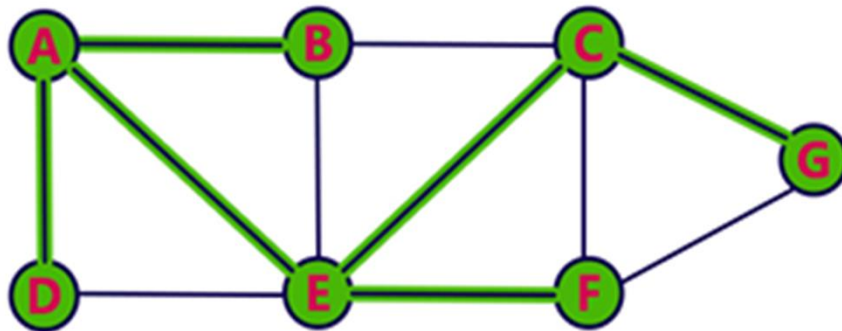


PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

GRAPH TRAVERSAL - BFS

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.



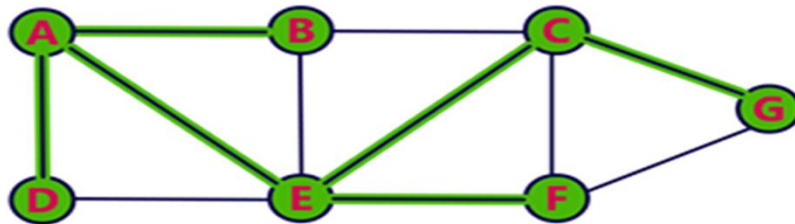
Queue



GRAPH TRAVERSAL - BFS

Step 8:

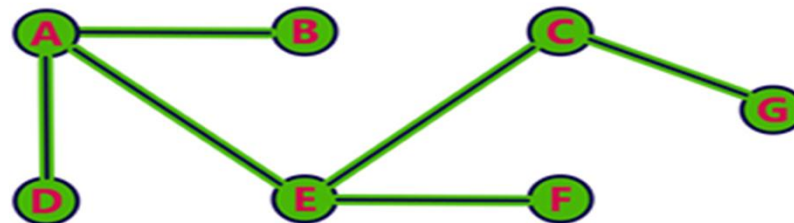
- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



Queue



-
- Queue became Empty. So, stop the BFS process.
 - Final result of BFS is a Spanning Tree as shown below...



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

COMPONENTS AND BICONNECTED COMPONENTS:

In graph theory, connected components and biconnected components are fundamental concepts that help in understanding the structure of graphs, especially in terms of connectivity and resilience to failures.

CONNECTED COMPONENTS

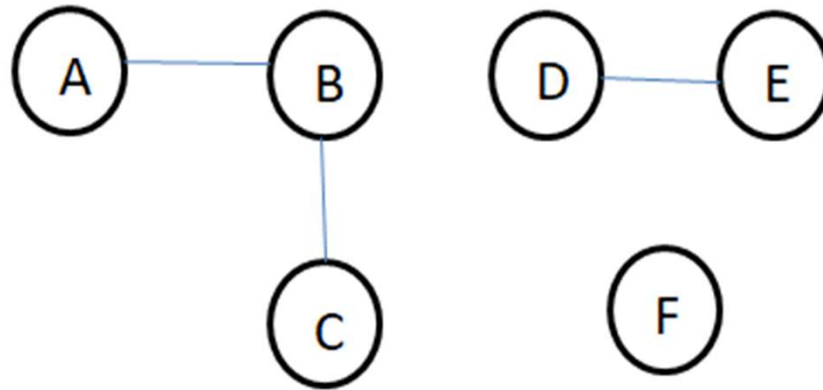
A connected component in an undirected graph is a maximal subgraph in which any two vertices are connected to each other by a path, and which is connected to no additional vertices in the rest of the graph.

Key Characteristics:

- Every vertex in a connected component is reachable from every other vertex within that component.
- If the graph is fully connected, it has exactly one connected component.

CONNECTED COMPONENTS

Example: Consider the following undirected graph:



In this graph:

- {A, B, C} forms one connected component.
- {D, E} forms another connected component.
- {F} forms the third connected component.

Connected Component:
Graph remains connected as long as there is at least one path between every pair of vertices.

In this example, the graph has three connected components.

CONNECTED COMPONENTS USING DFS

Algorithm:

1. Mark all vertices as **Not Visited**.
2. Initialize component number = 1.
3. Traverse all vertices.
4. If a vertex is not visited:
 - i. Print "Connected Component component_number:"
 - ii. Call DFS(vertex).
 - iii. Increment component number.
5. In DFS:
 - i. Mark current vertex as visited.
 - ii. Print the vertex.
 - iii. Visit all unvisited adjacent vertices recursively.

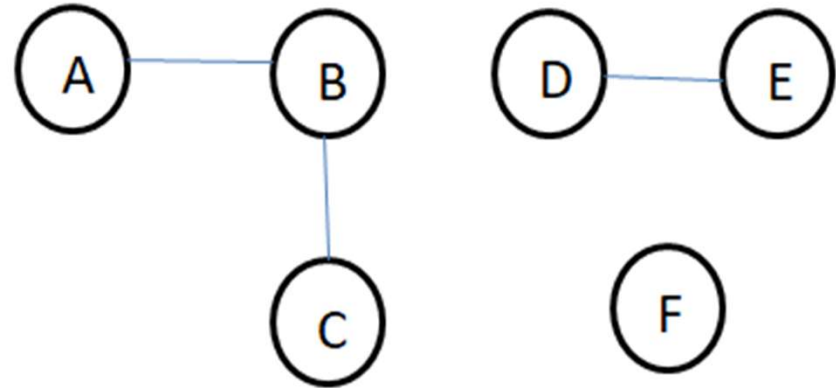
PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CONNECTED COMPONENTS USING DFS

Example Graph:

Adjacency List

A : B
B : A, C
C : B
D : E
E : D
F :



CONNECTED COMPONENTS USING DFS

DFS Traversal

Initially

Visited = { }

Vertex A

DFS(A)

A

└─B──C

Component 1

A B C

Visited

A B C

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

CONNECTED COMPONENTS USING DFS

Next unvisited vertex is D

DFS(D)

D

└─E

Component 2

D E

Visited

A B C D E

CONNECTED COMPONENTS USING DFS

Next unvisited vertex

DFS(F)

Component 3

F

Final Connected Components

$CC1 = \{A, B, C\}$

$CC2 = \{D, E\}$

$CC3 = \{F\}$

CONNECTED COMPONENTS USING BFS

Connected Components using BFS

Algorithm is almost the same.

Only DFS is replaced by BFS.

Algorithm

1. Mark all vertices as unvisited.
 2. Traverse every vertex.
 3. If unvisited
 - i. Create a queue.
 - ii. Push the vertex.
 - iii. Perform BFS.
 4. Continue until every vertex is visited.
- All visited vertices belong to one connected component.

CONNECTED COMPONENTS ALGORITHM

Connected_Components(G)

```
{  
  for each vertex v  
    flag[v] = -1;  
  count = 0;  
  for (v = 0; v < N; v++)  
  {  
    if (flag[v] == -1)  
    {  
      DFS(v);  
      count++;  
    }  
  }  
  printf("%d", count);  
}
```

DFS(v)

```
{  
  flag[v] = 1;  
  printf("%d", v);  
  
  for each adjacent node u of v  
  {  
    if (flag[u] == -1)  
    {  
      DFS(u);  
    }  
  }  
}
```

BICONNECTED COMPONENTS USING TARJAN'S ALGORITHM

Tarjan's Algorithm uses

- Discovery Time (disc[])
- Low Value (low[])
- Parent[]
- Stack of edges

BICONNECTED COMPONENTS USING TARJAN'S ALGORITHM

Time when a vertex is first visited.

$disc[u]$

Low Value

Lowest discovery time reachable from a vertex.

$low[u]$

Formula

$low[u] = \min(disc[u], disc[back\ edge], low[child])$

Conditions

Tree Edge

Parent $\rightarrow$ Child

Example

A $\rightarrow$ B

Back Edge

Child $\rightarrow$ Ancestor

Example

C $\rightarrow$ A

Articulation Point Condition

For non-root: $low[child] \geq disc[parent]$

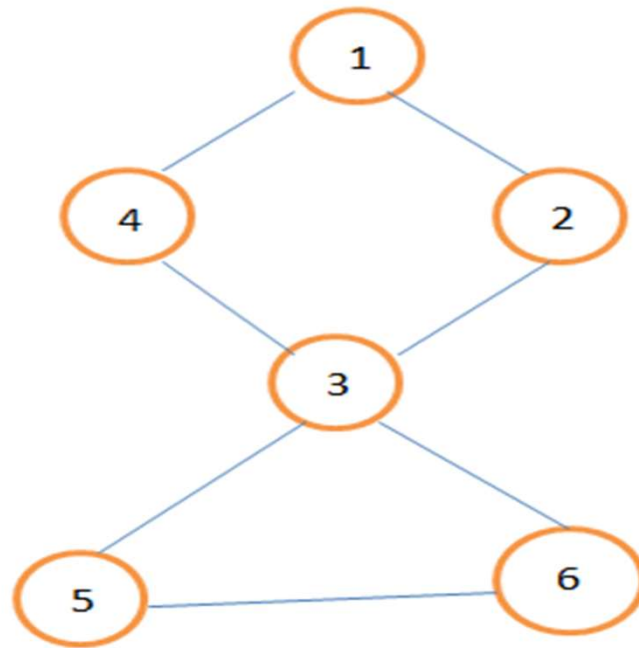
Root

More than one child

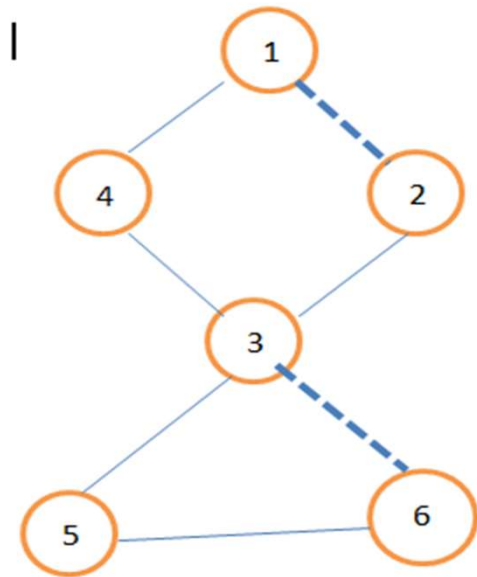
PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BICONNECTED COMPONENTS USING TARJAN'S ALGORITHM



BICONNECTED COMPONENTS USING TARJAN'S ALGORITHM



For each Vertex u , $L(u)$ is the lowest depth first number that can be reached from u using a path of descendants followed by at most one back edge

N	1	2	3	4	5	6
D	1	6	3	2	4	5
L	1	1	1	1	3	3

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

BICONNECTED COMPONENTS USING TARJAN'S ALGORITHM

$$L(u) = \min \{ \text{dfn}[u], \min (\{ L(v) / v \text{ is child of } u \}, \min \{ \text{dfn}(v) / (u,v) \text{ is a back edge} \}) \}$$

DIVIDE AND CONQUER

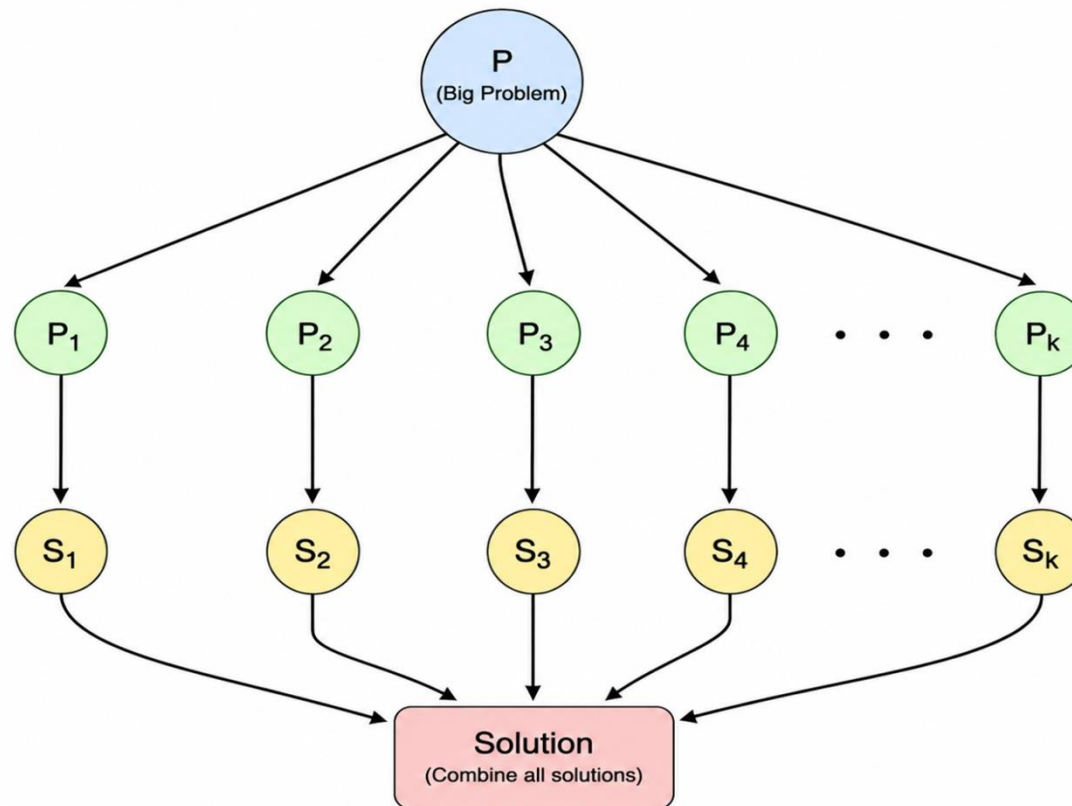
1. The General Method
2. Quick Sort
3. Merge Sort
4. Strassen's matrix multiplication
5. Convex Hull

DIVIDE AND CONQUER- GENERAL METHOD

The divide and conquer approach involves three steps at each level of recursion

1. **Divide:** Break the problem into several sub problems that are similar to original problem but smaller in size
2. **Conquer:** Solve the sub problems recursively. If the sub problem size is small enough, just solve the sub problem in a straight forward manner
3. **Combine:** Combine these solutions to create a solution to the original problem

DIVIDE AND CONQUER- GENERAL METHOD



GENERAL ALGORITHM:

DAC(P)

```
{
  if (small(P))
  {
    S(P);
  }
  else
  {
    divide P into  $P_1, P_2, P_3, \dots, P_k$ 
    apply DAC( $P_1$ ), DAC( $P_2$ ),  $\dots$ , DAC( $P_k$ )
    combine [DAC( $P_1$ ), DAC( $P_2$ ),  $\dots$ , DAC( $P_k$ )]
  }
}
```

```
T(n) =
{
  T(1),          when n is small
  aT(n/b) + f(n), when n is big
}
```

Where:

- a = number of subproblems
- n/b = size of each subproblem
- $f(n)$ = time required for divide + combine
- $T(n)$ = total time complexity of the Divide and Conquer algorithm.

PREPARED BY:

K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUICK SORT

1. Divide and Conquer principle is used in Quick Sort.
2. Quick Sort works by partitioning a given array: $A(p...r)$ into two non-empty subarrays: $A(p...q-1)$ and $A(q+1...r)$
3. It chooses the element as a pivot element, lower bound is the first index and upper bound is the last index.
4. Based on the pivot element chosen, the array is partitioned such that:
 - All greater elements are moved to the right side of the pivot.
 - All smaller elements are moved to the left side of the pivot.
5. Each partition is internally sorted recursively.

QUICK SORT ALGORITHM

```
QuickSort(l, h)
{
    if (l < h)
    {
        j = Partition(l, h);

        QuickSort(l, j);

        QuickSort(j + 1, h);
    }
}
```

The two recursive calls take: $T(n/2)$
when the pivot occurs at the middle.
If pivot occurs at first position: $T(0)$ and $T(n)$

QUICK SORT ALGORITHM

```
Partition(l, h)
{
    pivot = A[l];
    i = l;
    j = h;
    while (i < j)
    {
        do
        {
            i++;
        }
        while (A[i] <= pivot);
        do
        {
            j--;
        }
        while (A[j] >= pivot);
        if (i < j)
        {
            swap(A[i], A[j]);
        }
    }
    swap(A[l], A[j]);
    return j;
}
```

QUICK SORT ALGORITHM

Time Analysis : Running time depends on which element is chosen as the pivot.

Worst Case : The worst case occurs when the array is partitioned into: one subarray with $n - 1$ elements and the other with 0 elements.

$$\begin{aligned}T(n) &= T(n - 1) + T(0) + \Theta(n) \\&= T(n - 1) + \Theta(n)\end{aligned}$$

Therefore $T(n) = \Theta(n^2)$

QUICK SORT ALGORITHM

Best Case: Best case occurs when partition produces subproblems each of size: $n/2$

Therefore: $T(n) = 2T(n/2) + \Theta(n)$

$$T(n) = \Theta(n \log n)$$

Average Case: Average case is much closer to the best case. Suppose the partitioning algorithm produces a 9:1 proportional split.

The recurrence will be: $T(n) = T(9n/10) + T(n/10) + \Theta(n)$

Therefore: $T(n) = \Theta(n \log n)$

QUICK SORT EXAMPLE

Given array:

$A = [5, 3, 8, 1, 4, 6, 2, 7]$

We use:

1. Pivot = first element
2. i moves from left to right
3. j moves from right to left
4. If $A[i] > \text{pivot}$ and $A[j] < \text{pivot}$, swap them.
5. When $i \geq j$, swap the pivot with $A[j]$.
6. The pivot is then in its correct position.
7. Apply Quick Sort recursively to the left and right subarrays.

QUICK SORT EXAMPLE

Step 1: First Partition

Initial array:

5 3 8 1 4 6 2 7

↑

Pivot = 5

Set:

$l = 1$

$h = 8$

$i = 1$

$j = 8$

pivot = 5

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

QUICK SORT EXAMPLE

Move i

Start from the element after pivot. $A[2] = 3$

Since: $3 \leq 5$

move i.

$A[3] = 8$

Since: $8 > 5$

stop I

So: $i = 3$

QUICK SORT EXAMPLE

Move j

Start from the right.

$A[8] = 7$

Since: $7 \geq 5$

move j.

$A[7] = 2$

Since: $2 < 5$

stop j.

So: $j = 7$

QUICK SORT EXAMPLE

Now:

$i < j$

$3 < 7$

Therefore, swap: $A[3] \leftrightarrow A[7]$

Array becomes: 5 3 2 1 4 6 8 7

QUICK SORT EXAMPLE

Step 2: Continue First Partition

Now continue moving i.

Current:

$i = 3$

$j = 7$

Move i: $A[4] = 1$

Since: $1 \leq 5$

Continue $A[5] = 4$

QUICK SORT EXAMPLE

Since: $4 \leq 5$

Continue $A[6] = 6$

Since: $6 > 5$

stop.

Therefore: $i = 6$

Now move j. $A[6] = 6$

Since: $6 \geq 5$

move left. $A[5] = 4$

Since: $4 < 5$

stop

QUICK SORT EXAMPLE

Therefore: $j = 5$

Now:

$i \geq j$

$6 \geq 5$

So stop the partition process.

QUICK SORT EXAMPLE

Step 3: Place Pivot in Correct Position

Current array:

5 3 2 1 4 6 8 7

↑ ↑

pivot j

Swap: $A[1] \leftrightarrow A[5]$

Therefore: 4 3 2 1 5 6 8 7

Now 5 is in its correct position.

QUICK SORT EXAMPLE

So the array is divided into:

[4, 3, 2, 1] [5] [6, 8, 7]

Therefore, Quick Sort is applied recursively to:

Left subarray = [4, 3, 2, 1]

Right subarray = [6, 8, 7]

QUICK SORT EXAMPLE

Step 4: Quick Sort on Left Subarray

Consider: [4, 3, 2, 1]

Choose first element as pivot:

Pivot = 4

4 3 2 1

↑

pivot

QUICK SORT EXAMPLE

Move i. $3 \leq 4$

move i. $2 \leq 4$

move i. $1 \leq 4$

move i. There are no more elements.

Move j

Starting from the right: $1 < 4$

So j stops at 1.

Now: $i \geq j$

Therefore, swap pivot 4 with 1.

QUICK SORT EXAMPLE

Therefore, swap pivot 4 with 1.

1 3 2 4

Now 4 is in its correct position.

Array becomes:

[1, 3, 2] [4]

So continue Quick Sort on: [1, 3, 2]

QUICK SORT EXAMPLE

Step 5: Quick Sort on [1, 3, 2]

Choose first element: Pivot = 1

1 3 2

↑

Pivot

Since both remaining elements are greater than 1:

$3 > 1$

$2 > 1$

The pivot 1 remains at the beginning.

QUICK SORT EXAMPLE

Therefore:

[1] [3, 2]

Now sort: [3, 2]

Step 6: Quick Sort on [3, 2]

Choose: Pivot = 3

3 2

↑

Pivot

Since: $2 < 3$

swap pivot with 2.

QUICK SORT EXAMPLE

Result: 2 3

Therefore:

[1] [2, 3] [4]

The complete left side is now:

1 2 3 4

QUICK SORT EXAMPLE

Step 7: Quick Sort on Right Subarray

From Step 3, the right subarray was: [6, 8, 7]

Choose first element: Pivot = 6

6 8 7

↑

Pivot

Both elements are greater than 6:

$8 > 6$

$7 > 6$

Therefore, 6 remains in its correct position.

We get: [6] [8, 7]

QUICK SORT EXAMPLE

Step 8: Quick Sort on [8, 7]

Choose: Pivot = 8

8 7

↑

Pivot

Since: $7 < 8$

swap: $8 \leftrightarrow 7$

Result: 7 8

Therefore, the right side becomes: 6 7 8

QUICK SORT EXAMPLE

Step 9: Final Sorted Array

Combining the left side, pivot 5, and right side:

1 2 3 4 5 6 7 8

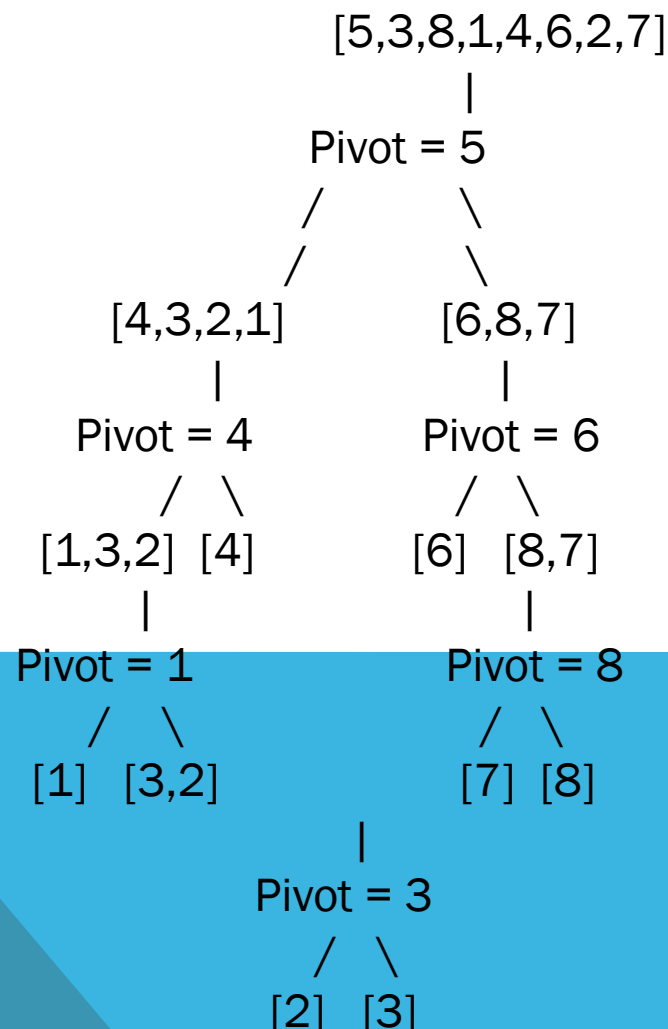
Therefore: Sorted Array

$A = [1, 2, 3, 4, 5, 6, 7, 8]$

QUICK SORT EXAMPLE

Recursion Tree

The Quick Sort recursion can be represented as:



MERGE SORT

1. Merge Sort is an example of the Divide and Conquer algorithm.
2. It is based on the idea of breaking down a list into several sublists until each sublist consists of a single element.
3. Merge these sublists in a manner that results in a sorted list.

MERGE SORT - ALGORITHM

```
MergeSort(l, h)
{
    if (l < h)
    {
        mid = (l + h) / 2;

        MergeSort(l, mid);

        MergeSort(mid + 1, h);

        Merge(l, mid, h);
    }
}
```

MERGE SORT - ALGORITHM

Time taken:

MergeSort(l, mid) $\rightarrow T(n/2)$

MergeSort(mid + 1, h) $\rightarrow T(n/2)$

Merge(l, mid, h) $\rightarrow n$

MERGE SORT - ALGORITHM

```
Merge(l, mid, h)
{
    i = 1;
    j = 1;
    k = 1;

    while (i <= m && j <= n)
    {
        if (A[i] < B[j])
        {
            C[k++] = A[i++];
        }
        else
        {
            C[k++] = B[j++];
        }
    }

    for (; i <= m; i++)
        C[k++] = A[i];

    for (; j <= n; j++)
        C[k++] = B[j];
}
```

MERGE SORT - ANALYSIS

Analysis

$$T(n) = \begin{cases} 1, & n = 1 \\ 2T(n/2) + n, & n > 1 \end{cases}$$

Therefore:

$$T(n) = 2T(n/2) + n$$

MERGE SORT - ANALYSIS

Substitute: $T(n/2) = 2T(n/4) + n/2$

Therefore:

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$= 4T(n/4) + n + n$$

$$= 4T(n/4) + 2n$$

After k iterations: $T(n) = 2^k T(n/2^k) + kn$

Assume: $n / 2^k = 1$

Therefore:

$$n = 2^k$$

$$k = \log_2 n$$

MERGE SORT - ANALYSIS

Hence:

$$T(n) = nT(1) + n \log n$$

$$T(n) = n + n \log n$$

Therefore:

$$T(n) = \Theta(n \log n)$$

Note: The best, average and worst case of Merge Sort is:

$$\Theta(n \log n)$$

MERGE SORT EXAMPLE

Consider: [724, 521, 2, 98, 529, 31, 189, 451]

Step 1: Split the selected array based on the middle element

[724, 521, 2, 98] [529, 31, 189, 451]

Left subarray

[724, 521] [2, 98]

Further divide:

[724] [521] [2] [98]

Merge:

[521, 724] and: [2, 98]

Merge them: [2, 98, 521, 724]

MERGE SORT EXAMPLE

Right subarray

[529, 31] [189, 451]

Further divide:

[529] [31] [189] [451]

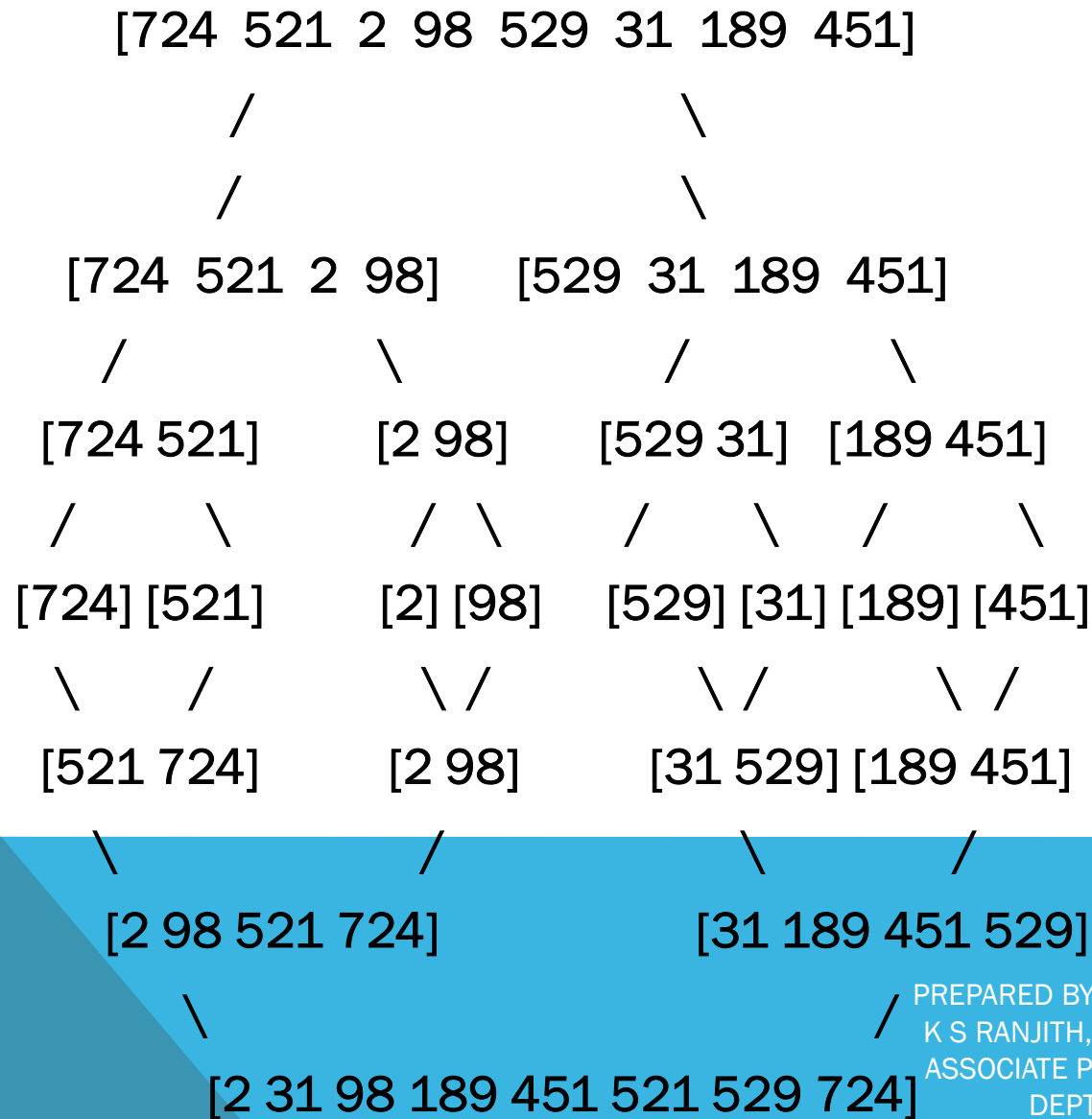
Merge: [31, 529] and: [189, 451]

Merge them: [31, 189, 451, 529]

Finally merge both sorted subarrays:

[2, 31, 98, 189, 451, 521, 529, 724]

MERGE SORT EXAMPLE



PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

STRASSEN'S MATRIX MULTIPLICATION

- Let us consider two matrices X and Y and calculate the resultant matrix Z by multiplying X and Y.

- **Iterative Method**

First we will see iterative (naive) method and its time complexity. Here we calculate: $Z = X \times Y$

Algorithm

mm(X, Y, Z)

for i = 1 to p do

 for j = 1 to r do

$Z[i,j] = 0$

 for k = 1 to q do

$Z[i,j] = Z[i,j] + X[i,k] * Y[k,j]$

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

STRASSEN'S MATRIX MULTIPLICATION

Complexity

There are three for loops in this algorithm and each one is independent of each other.

Hence the algorithm takes: $O(n^3)$

STRASSEN'S MATRIX MULTIPLICATION

1. In this context, using Strassen's matrix multiplication algorithm, the time consumption can be improved a little bit.
2. Strassen's matrix multiplication can be performed only on square matrices where n is a power of 2.
3. Both the matrices are of order: $n \times n$

STRASSEN'S MATRIX MULTIPLICATION

Divide Matrices : Divide A, B and C into four: $n/2 \times n/2$

For example, when $n = 2$:

$$A = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix}$$

$$B = \begin{vmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{vmatrix}$$

$$C = \begin{vmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{vmatrix}$$

When $n = 4$, the matrices are divided into four 2×2 submatrices.

STRASSEN'S MATRIX MULTIPLICATION

Algorithm mm(A, B, n)

```
{  
  if (n <= 2)  
  {  
    C11 = A11 * B11 + A12 * B21;  
  
    C12 = A11 * B12 + A12 * B22;  
  
    C21 = A21 * B11 + A22 * B21;  
  
    C22 = A21 * B12 + A22 * B22;  
  }  
  else  
  {  
    mid = n/2;
```

```
    mm(A11, B11, n/2) + mm(A12, B21, n/2)  
  
    mm(A11, B12, n/2) + mm(A12, B22, n/2)  
  
    mm(A21, B11, n/2) + mm(A22, B21, n/2)  
  
    mm(A21, B12, n/2) + mm(A22, B22, n/2)  
  }  
}
```

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET

STRASSEN'S MATRIX MULTIPLICATION

- Computing each entry in the product takes n multiplications and there are n^2 entries for a total of: $O(n^3)$

Therefore: $T(n) = 8T(n/2) + n^2$

Solving, we get: $O(n^3)$

1. Strassen devised a better method which has the same basic method as multiplication of long integers.
2. The main idea is to save one multiplication on a small problem and then use a recursion.

STRASSEN'S MATRIX MULTIPLICATION

Strassen's Algorithm for Matrix Multiplication Involves

Step 1

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

STRASSEN'S MATRIX MULTIPLICATION

Step 2

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

STRASSEN'S MATRIX MULTIPLICATION- ANALYSIS

- It is therefore possible to multiply two 2×2 matrices using only seven scalar multiplications.
- Let $T(n)$ be the time needed to multiply two $n \times n$ matrices by recursive use of the equation.

STRASSEN'S MATRIX MULTIPLICATION- ANALYSIS

Strassen's Matrix Multiplication- ANALYSIS

$$T(n) = 7T(n/2) + f(n)$$

where: $f(n) = O(n^2)$

- If we compute the above using the Back Substitution method or by Master's Theorem method, we get:
 - $T(n) = O(n^{2.81})$
- Therefore, Strassen's matrix multiplication reduces the time complexity from: $O(n^3)$ to: $O(n^{2.81})$

THANK YOU

PREPARED BY:
K S RANJITH, M.E (Ph.D),
ASSOCIATE PROFESSOR,
DEPT. OF AI&DS,
MTIET